Оглавление

ВВЕДЕНИЕ **4**

Глава 1. Анализ предметной области **7**

1.1 Характеристика процесса учёта технических средств **7**

1.2 Сравнительный анализ существующих решений **8**

1.3 Обоснование необходимости разработки **15**

1.4 Требования к разрабатываемой системе **16**

Глава 2. Проектирование информационной системы **19**

2.1 Функциональная модель **19**

2.2 Информационная модель **22**

2.3 Моделирование бизнес-процессов (AS-IS / TO-BE) **30**

2.4 Архитектура системы **32**

2.5 Динамическая модель системы **35**

Глава 3. Описание модулей приложения **39**

3.1 Структура проекта **39**

3.2 Модуль базы данных (database.py) **40**

3.3 Модуль серверной части (app.py) **43**

3.4 Модуль интерфейса (HTML-шаблоны) **46**

3.5 Модуль генерации QR-кодов **48**

3.6 Модуль аналитики и рейтингов **49**

3.7 Модуль учёта расходных материалов (ТМЦ) **50**

Глава 4. Описание интерфейса компонентов **51**

4.1 Главная страница (дашборд) **51**

4.2 Справочник оборудования **52**

4.3 Добавление и редактирование оборудования **53**

4.4 Детальный просмотр оборудования **53**

4.5 Расширенная аналитика (6 рейтингов) **54**

4.6 Отчёты **55**

4.7 Управление расходными материалами **56**

4.8 Совместимость комплектующих **56**

4.9 QR-коды **57**

4.10 Поиск **57**

4.11 Навигация 57
Тестирование системы 58
Заключение 61
СПИСОК ЛИТЕРАТУРЫ 64
ПРИЛОЖЕНИЕ А 66
ПРИЛОЖЕНИЕ Б. Скриншоты интерфейса 82

ВВЕДЕНИЕ

Актуальность темы. Современное развитие организаций характеризуется постоянным ростом количества используемых технических средств: персональных компьютеров, ноутбуков, серверного и сетевого оборудования, периферийных устройств. Эффективное управление этим парком техники становится критически важной задачей, поскольку от достоверности учёта зависят финансовые затраты, производительность сотрудников и информационная безопасность организации [4, с. 112]

Традиционные методы учёта с использованием бумажных журналов и электронных таблиц Excel перестают отвечать современным требованиям. Они характеризуются разрозненностью данных, высокой вероятностью ошибок ручного ввода, отсутствием оперативного контроля перемещений и трудоёмкостью проведения инвентаризаций.

Существующие специализированные ITAM-системы (IT Asset Management) часто обладают избыточным функционалом, высокой стоимостью внедрения и требуют длительного обучения персонала. В связи с этим актуальной является задача разработки собственной информационной системы, ориентированной на конкретные потребности организации, позволяющей оперативно вести учёт технических средств, контролировать их перемещения и обслуживание, а также формировать необходимую отчётность.

Цель работы - Повышение эффективности учёта технических средств путём разработки и внедрения информационной системы, обеспечивающей централизованное хранение данных об оборудовании, контроль перемещений и аналитическую поддержку принятия решений.

.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и существующих решений.
2. Выявить недостатки текущего процесса учёта.
3. Сформулировать функциональные и нефункциональные требования к системе.
4. Выбрать архитектуру и технологии разработки.
5. Спроектировать структуру базы данных.
6. Разработать пользовательский интерфейс.
7. Реализовать программный прототип системы.
8. Провести тестирование и оценить экономическую эффективность.

Объект исследования - процесс учёта технических средств в организации.

Что именно изучается: совокупность действий по фиксации, хранению и контролю информации об оборудовании (ПК, ноутбуки, серверы, сетевое оборудование, периферия)

Предмет исследования - методы и средства автоматизации учёта технических средств, ориентированные на потребности IT-отдела организации.

Что разрабатывается: информационная система, объединяющая базу данных, веб-интерфейс, аналитические инструменты и средства идентификации (QR-коды)

Новизна разрабатываемой системы заключается в следующем:

1. Интеграция веб-интерфейса, SQLite, QR-кодов и аналитического дашборда в едином решении для учёта технических средств.
2. Автоматическая генерация QR-кодов для каждого устройства с возможностью печати и сканирования через обычный смартфон, что позволяет проводить мобильную инвентаризацию без дорогостоящего оборудования.
3. Разработан дашборд с графиками распределения по типам, динамики добавления и топом проблемных устройств (на основе количества поломок).
4. Реализованы расширенные функции учёта: отслеживание ремонтов, лицензий программного обеспечения, профилактических работ, расходных материалов (ТМЦ) и совместимости комплектующих.
5. Система является полностью бесплатной, имеет открытый код и легко адаптируется под нужды конкретной организации силами IT-отдела без привлечения платных программистов.

Практическая значимость работы. Разработанная система может быть использована IT-отделами организаций для учёта компьютерной и оргтехники, сетевого оборудования, контроля перемещений, планирования ремонтов и профилактики, учёта лицензий и расходных материалов. Система готова к внедрению, не требует финансовых затрат и специального обучения персонала.

Глава 1. Анализ предметной области

1.1 Характеристика процесса учёта технических средств

Объектом автоматизации выступает процесс учёта технических средств в организации. Под **техническими средствами** понимается совокупность оборудования, используемого для обеспечения деятельности организации, включая:

- **Вычислительную технику:** персональные компьютеры (системные блоки, моноблоки), ноутбуки, планшеты.
- **Сетевое оборудование:** коммутаторы, маршрутизаторы, точки доступа Wi-Fi, серверы.
- **Периферийные устройства:** мониторы, принтеры, МФУ, сканеры, проекторы.
- **Вспомогательное оборудование:** источники бесперебойного питания, внешние накопители, гарнитуры.

В ходе анализа выявлены следующие **участники процесса учёта**:

Таблица 1 - Роли и функции участников процесса учёта технических средств

Роль	Функции
Системный администратор / IT-менеджер	Внесение данных о новом оборудовании, назначение ответственных лиц, контроль перемещений, формирование отчётов

Роль	Функции
Материально-ответственное лицо (МОЛ)	Получение оборудования под отчёт, подтверждение перемещений, участие в инвентаризации
Руководитель подразделения	Согласование заявок на оборудование, контроль использования
Бухгалтер	Сверка данных инвентаризации, учёт амортизации

В настоящее время учёт ведётся с использованием **Microsoft Excel**. При поступлении нового оборудования сотрудник IT-отдела вносит информацию в таблицу, затем распечатывает акт приёма-передачи. При перемещении оборудования между сотрудниками информация обновляется вручную, что часто приводит к рассинхронизации данных.

1.2 Сравнительный анализ существующих решений

На рынке представлен ряд программных продуктов для учёта технических средств. Проведён сравнительный анализ трёх категорий решений [5, с. 78].

1.2.1. Excel и бумажный учёт

Таблица 2 — Сравнительный анализ Excel и бумажного учёта

Критерий	Оценка
Стоимость	Низкая (требуется только лицензия MS Office)
Функционально-сть	Ограниченная (только хранение, нет автоматических связей)
Надежность	Низкая (риск потери данных, ошибок ввода)
Скорость работы	Низкая при увеличении объёма данных
Контроль версий	Отсутствует

1.2.2. Коробочные ITAM-системы (SimpleOne, ИнфраМенеджер, Колибри-АРМ)

Таблица 3 — Сравнительный анализ коробочных ITAM-систем

Критерий	Оценка
Стоимость	Высокая (лицензии от 100 000 руб.)
Функциональность	Широкая, часто избыточная

Критерий	Оценка
Надежность	Высокая
Скорость внедрения	Длительная (от 1 до 3 месяцев)
Возможность доработки	Ограничена, требует привлечения вендора

1.2.3. Open-source решения (GLPI, OCS Inventory)

Таблица 4 — Сравнительный анализ Open-source решений

Критерий	Оценка
Стоимость	Бесплатно (требуется настройка)
Функциональность	Средняя, требует доработок
Надежность	Средняя
Сложность настройки	Высокая (требуется специалист)
Поддержка	Силами сообщества

1.2.4. Сравнительный анализ с 1С:ERP

В ходе исследования также проведён сравнительный анализ с промышленной ERP-системой 1С:ERP, которая часто используется для учёта основных средств.

Таблица 5 — Сравнительный анализ разработанной ИС и 1С:ERP

Критерий	1С:ERP	Разработанная ИС
QR-коды	Нет в базовой версии, требует доработок за 150 000+ руб.	Есть «из коробки», бесплатно
Мобильная инвентаризация	Требует ТСД за 50 000 руб. и настройки обмена данными	Через любой смартфон, бесплатно
Гибкость и кастомизация	Низкая, требует платного программиста (от 2000 руб./час)	Высокая, изменения вносятся за 5-10 минут

Критерий	1С:ERP	Разработанная ИС
Стоимость внедрения	От 500 000 руб. (лицензия + внедрение)	0 руб.
Учёт лицензий ПО	Есть, но сложный, интегрирован с бухгалтерией	Простой и наглядный, с контролем истечения срока
Аналитика	Громоздкие отчёты, требующие обучения	Наглядные дашборды с графиками

Выводы по анализу:

- Excel-решения не обеспечивают необходимого уровня контроля и надежности.
- Коммерческие ITAM-системы функционально избыточны для большинства малых и средних организаций и имеют высокую стоимость.
- Open-source решения требуют значительных трудозатрат на настройку и адаптацию.

- Промышленные ERP-системы (1С:ERP) дороги и сложны в настройке, а также не предоставляют современных инструментов мобильной инвентаризации (QR-коды) «из коробки».

Таким образом, обоснована необходимость собственной разработки.

1.2.5 Сравнительный анализ решений и преимущества разработанной ИС

Таблица 6 — Сравнительный анализ решений для учёта технических средств

Критерий	Excel / Бумага	ITAM-системы	ERP-системы (1С:ERP / SAP ERP)	Разработанная ИС
Стоимость внедрения	Низкая	Высокая (от 100 т.р.)	1С: от 500 т.р. / SAP: от 3 млн руб.	0 руб.
Единая база данных	Нет	Да	Да	Да (SQLite)
Автоматизация	Нет	Частично	Полная	Полная
Удобство интерфейса	Низкое	Среднее	Среднее (требует обучения)	Высокое (веб)
Возможность доработки	-	Сложно (вендор)	Очень сложно (дорого)	Лёгкая (Python)

QR-коды для устройств	Нет	Редко	Нет в базовой версии	Есть
Аналитика и графики	Нет	Частично	Громоздкие отчёты	Дашборд с графиками
Экспорт в Excel	Нет	Есть	Есть	Есть
Учёт лицензий ПО	Нет	Частично	Сложный, с бухгалтерией	Простой, с контролем
Учёт расходных материалов	Нет	Редко	Требует доработки	Есть

Преимущества разработанной ИС:

1. **Низкая стоимость внедрения** - все инструменты бесплатны
2. **Простота использования** - веб-интерфейс, интуитивно понятно
3. **Гибкость** - код открыт, легко добавлять новые функции
4. **QR-коды** - быстрая идентификация оборудования
5. **Аналитика** - дашборд с графиками и статистикой
6. **Экспорт данных** - выгрузка в Excel для отчётности

7. **Статистика в реальном времени** - общее количество оборудования, число единиц в эксплуатации, на складе и в ремонте.
8. **Совместимость комплектующих -** система содержит рекомендации по подбору совместимых запчастей
9. **Учёт расходных материалов (ТМЦ)** -ведение остатков картриджей, бумаг.
10. **Учёт ремонтов и поломок -** регистрация всех случаев неисправностей с фиксацией даты, описания и стоимости ремонта.

1.3 Обоснование необходимости разработки

На основе проведённого анализа сформулированы **проблемы текущего процесса учёта**:

1. **Отсутствие единой базы данных** - информация хранится в разрозненных файлах, что приводит к противоречиям.
2. **Низкая достоверность данных** - ручной ввод сопровождается ошибками и дублированием.
3. **Трудоёмкость поиска** - определение местонахождения конкретного устройства может занимать до 30 минут.
4. **Сложность инвентаризации** - сверка фактического наличия с учётными данными занимает несколько дней.
5. **Отсутствие контроля перемещений** - не всегда фиксируется факт передачи оборудования между сотрудниками.
6. **Нет напоминаний об обслуживании** - пропускаются сроки замены картриджей, поверки оборудования, истечения гарантии и лицензий.

Необходимость разработки собственной системы обусловлена:

- потребностью в решении, точно соответствующем процессам организации;
- желанием минимизировать затраты на внедрение и сопровождение;
- необходимостью оперативной доработки функционала при изменении требований;
- потребностью в современных инструментах (QR-коды, мобильная инвентаризация, наглядная аналитика).

1.4 Требования к разрабатываемой системе

В рамках данной работы требуется разработать информационную систему, обеспечивающую следующие **функциональные требования** (FR) [1]:

Таблица 7 — Функциональные требования к информационной системе (FR)

ID	Требование
FR-01	Ведение единого справочника технических средств с возможностью добавления, редактирования, удаления записей
FR-02	Учёт инвентарных и серийных номеров, местоположения, статуса, материально-ответственного лица
FR-03	Поддержка классификации технических средств по типам (ПК, ноутбуки, сетевое оборудование и т.д.)

ID	Требование
FR-04	Регистрация перемещений оборудования с сохранением истории
FR-05	Ведение истории ремонтов и обслуживания
FR-06	Учёт лицензий программного обеспечения с контролем истечения срока действия
FR-07	Учёт расходных материалов (ТМЦ) с отслеживанием остатков и минимального уровня
FR-08	Поддержка совместимости комплектующих для оборудования
FR-09	Формирование отчётов: инвентаризационная ведомость, отчёт по перемещениям, отчёт по ремонтам, отчёт по лицензиям, график профилактики
FR-10	Поиск и фильтрация по различным критериям
FR-11	Генерация QR-кодов для каждого устройства и возможность их печати
FR-12	Экспорт данных в формате Excel

ID	Требование
FR-13	Разграничение прав доступа (администратор, IT-специалист, МОЛ)
FR-14	Аналитический дашборд с графиками и рейтингами (топ проблемных устройств, самые старые, требующие профилактики, с истекающей гарантией, часто перемещаемые, самые дорогие)

Нефункциональные требования (NFR):

Таблица 8 — Нефункциональные требования к информационной системе (NFR)

ID	Требование
NFR-01	Время отклика системы не более 2 секунд
NFR-02	Поддержка одновременной работы до 50 пользователей
NFR-03	Кроссплатформенность (работа через веб-браузер)
NFR-04	Резервное копирование данных (файл базы данных)
NFR-05	Интуитивно понятный интерфейс, адаптированный для работы IT-отдела

Глава 2. Проектирование информационной системы

Выбор средств разработки:

При выборе средств разработки учитывались требования:

- простота развёртывания (запуск одной командой)
- кроссплатформенность (работа через браузер)
- отсутствие платного ПО
- гибкость для доработок

Стек технологий:

- Python 3.11 + Flask
- SQLite
- HTML5 / CSS3 / Bootstrap 5
- qrcode + Pillow (QR-коды)
- openpyxl (Excel)

Среда разработки: IDLE

2.1 Функциональная модель



Рис 1. Дерево целей-функций

Функциональная модель предназначена для изучения особенностей работы (функционирования) системы и её назначения во взаимосвязи с внутренними и внешними элементами [9, с. 112].

Для разрабатываемой информационной системы построена диаграмма целей-функций, которая декомпозирует цели и функции приложения и представляет их наглядно.

2.1.1. Дерево целей-функций

Вершиной дерева является главная цель - «Автоматизация учёта технических средств в организации». Для её достижения выделены следующие подцели:

- **Управление справочником оборудования** - включает функции добавления, редактирования, удаления и просмотра информации об оборудовании.
- **Контроль перемещений** - включает функции регистрации выдачи оборудования, возврата на склад, передачи между сотрудниками с сохранением истории.
- **Управление ремонтами и обслуживанием** - включает функции регистрации поломок, планирования профилактических работ, учёта затрат на ремонт.
- **Управление лицензиями ПО** - включает функции добавления лицензий, контроля истечения срока действия, уведомлений.
- **Управление расходными материалами** - включает функции учёта остатков, списания, контроля минимального уровня.
- **Поиск и аналитика** - включает функции поиска по оборудованию, формирования отчётов, визуализации данных на дашборде.

- **Экспорт данных** - включает функцию выгрузки информации в формате Excel.
- **Идентификация оборудования** - включает функцию генерации QR-кодов для каждого устройства.
- **Администрирование** - включает функции управления пользователями и разграничения прав доступа.

2.1.2. Диаграмма прецедентов (Use Case Diagram)

На диаграмме прецедентов выделены следующие роли:



Рис 2. Диаграмма прецедентов

- **Администратор** - имеет полный доступ ко всем функциям системы: управление пользователями, настройка ролей, просмотр любых данных.
- **IT-специалист** - отвечает за внесение данных об оборудовании, регистрацию перемещений, добавление ремонтов и лицензий, формирование отчётов.
- **Материально-ответственное лицо (МОЛ)** - просматривает закреплённое за ним оборудование, подтверждает факт получения/передачи.
- **Наблюдатель** - имеет доступ только к просмотру информации.

Основные прецеденты (варианты использования):

1. Авторизация в системе
2. Добавление/редактирование/удаление оборудования
3. Привязка QR-кода к оборудованию
4. Регистрация перемещения оборудования
5. Добавление записи о ремонте
6. Добавление лицензии ПО
7. Списание расходного материала
8. Поиск оборудования по инвентарному номеру или наименованию
9. Просмотр аналитического дашборда
10. Формирование отчётов (поломки, лицензии, профилактика)
11. Экспорт данных в Excel
12. Управление пользователями

2.2 Информационная модель

Информационная модель представляет собой совокупность UML-диаграмм классов и схем базы данных.

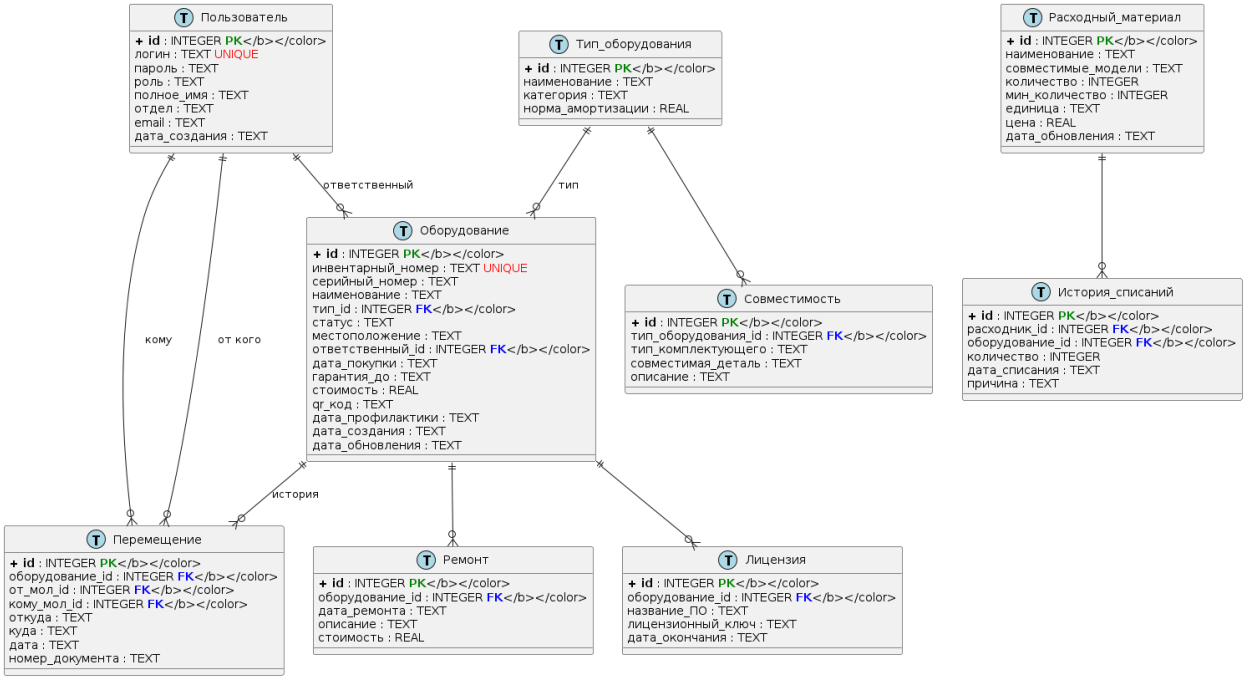


Рис 3. Диаграмма классов – Информационная модель базы данных

2.2.1. Логическая модель данных

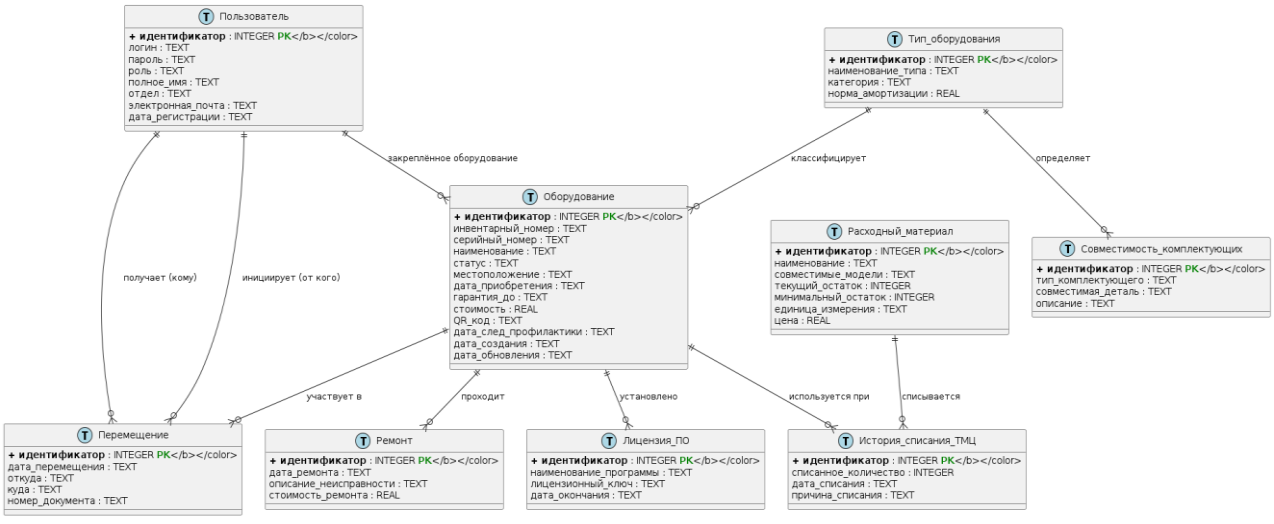


Рис 4. Диаграмма классов - Логическая модель (Диаграмма классов с сущностями и их атрибутами)

Основные сущности базы данных [13, с. 45]:

Таблица 9 - Основные сущности базы данных

Сущность	Атрибуты
User (пользователь)	id, username, password, role, full_name, department, email, created_at
EquipmentType (тип оборудования)	id, name, category, depreciation_rate
Equipment (оборудование)	id, inventory_number, serial_number, name, type_id, status, location, responsible_id, purchase_date, warranty_end, cost, qr_code, next_maintenance_date, created_at, updated_at
Movement (перемещение)	id, equipment_id, from_mol_id, to_mol_id, from_location, to_location, date, document_number
Repair (ремонт)	id, equipment_id, repair_date, description, cost
License (лицензия)	id, equipment_id, software_name, license_key, expiration_date

Сущность	Атрибуты
Consumable (расходный материал)	id, name, compatible_models, quantity, min_quantity, unit, price
Compatibility (совместимость)	id, equipment_type_id, component_type, compatible_part, description

Связи между сущностями:

- Equipment → EquipmentType (N:1)
- Equipment → User (N:1)
- Movement → Equipment (N:1)
- Movement → User (N:1, дважды: от кого и кому)
- Repair → Equipment (N:1)
- License → Equipment (N:1)
- Compatibility → EquipmentType (N:1)

2.2.2. Физическая схема базы данных

База данных реализована на СУБД SQLite и состоит из восьми основных таблиц, связанных между собой внешними ключами.

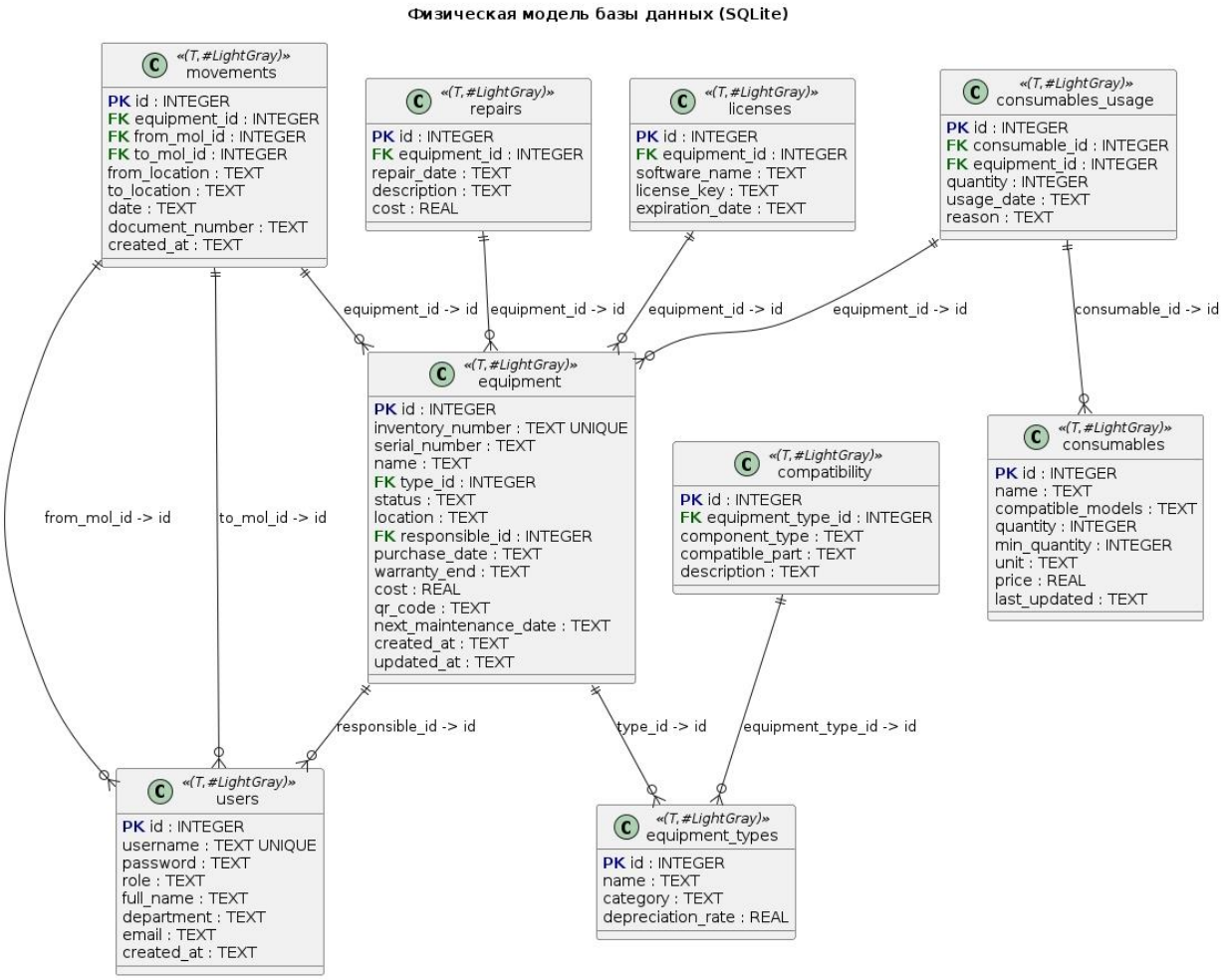


Рис 5. Диаграмма классов – Физическая модель.

Таблица users (пользователи) предназначена для хранения учётных записей сотрудников, имеющих доступ к системе. Содержит следующие поля:

- Id - уникальный идентификатор пользователя (первичный ключ);
- username - логин для входа в систему (уникальный);
- password - пароль в открытом виде (для учебного проекта);
- role - роль пользователя (admin, it_user, mol);
- full_name - полное имя пользователя;
- department - подразделение;
- email - адрес электронной почты;

- created_at - дата и время создания записи.

Таблица equipment_types (типы оборудования) служит справочником для классификации технических средств. Включает поля:

- id - идентификатор типа (первичный ключ);
- name - наименование типа (например, «ПК», «Ноутбук», «МФУ»);
- category - категория оборудования («Компьютеры», «Периферия», «Сетевое»);
- depreciation_rate - норма амортизации в процентах.

Таблица equipment (оборудование) является центральной и хранит информацию о каждом техническом средстве. Основные поля:

- id - уникальный идентификатор оборудования (первичный ключ);
- inventory_number - инвентарный номер (уникальный, обязательный);
- serial_number - серийный номер производителя;
- name - наименование оборудования;
- type_id - внешний ключ, ссылающийся на таблицу equipment_types;
- status - текущий статус (на складе, в эксплуатации, в ремонте, списано);
- location - местоположение (кабинет, склад, серверная);
- responsible_id - внешний ключ, ссылающийся на таблицу users (МОЛ);
- purchase_date - дата приобретения;

- warranty_end - дата окончания гарантии;
- cost - стоимость приобретения;
- qr_code - сгенерированный QR-код в формате base64;
- next_maintenance_date - планируемая дата следующей профилактики;
- created_at, updated_at - служебные поля даты создания и последнего изменения.

Таблица movements (перемещения) хранит историю всех передач оборудования между сотрудниками или складами. Поля:

- id - идентификатор записи (первичный ключ);
- equipment_id - внешний ключ на таблицу equipment;
- from_mol_id, to_mol_id - внешние ключи на таблицу users (от кого и кому передано);
- from_location, to_location - место отправления и назначения;
- date - дата перемещения;
- document_number - номер сопроводительного документа.

Таблица repairs (ремонты и поломки) фиксирует все случаи обслуживания оборудования. Содержит:

- id - идентификатор записи (первичный ключ);
- equipment_id - внешний ключ на оборудование;
- repair_date - дата ремонта;
- description - описание неисправности и выполненных работ;
- cost - стоимость ремонта.

Таблица licenses (лицензии ПО) предназначена для учёта программного обеспечения, установленного на оборудовании. Поля:

- id - идентификатор лицензии;
- equipment_id - внешний ключ на оборудование;
- software_name - наименование программного продукта;
- license_key - лицензионный ключ;
- expiration_date - дата окончания срока действия лицензии.

Таблица consumables (расходные материалы) ведёт учёт ТМЦ, используемых при эксплуатации оборудования (картриджи, тонеры, бумага и т.д.). Включает:

- id - идентификатор расходного материала;
- name - наименование;
- compatible_models - совместимые модели оборудования;
- quantity - текущий остаток;
- min_quantity - минимально допустимый остаток для уведомления;
- unit - единица измерения (шт., пачка, услуга);
- price - цена за единицу;
- last_updated - дата последнего обновления остатка.

Таблица compatibility (совместимость комплектующих) содержит рекомендации по подбору запчастей для различных типов оборудования. Поля:

- id - идентификатор записи;
- equipment_type_id - внешний ключ на тип оборудования (ПК, ноутбук и т.д.);

- component_typeь - тип комплектующего (ОЗУ, SSD, HDD, аккумулятор);
- compatible_part - наименование совместимой детали;
- description - дополнительная информация (производители, характеристики).

Связи между таблицами организованы следующим образом:

- Таблица equipment связана с equipment_types(N:1) и с users (N:1);
- Таблица movements связана с equipment (N:1) и дважды с users (от кого, кому);
- Таблицы repairs и licenses связаны с equipment(N:1);
- Таблица compatibility связана с equipment_types (N:1).

2.3 Моделирование бизнес-процессов (AS-IS / TO-BE)

Для анализа эффективности разрабатываемой системы смоделированы два бизнес-процесса: «как есть» (AS-IS) и «как будет» (TO-BE) [7, с. 89].

2.3.1. AS-IS (текущий процесс учёта)

Блок-схема текущего процесса:

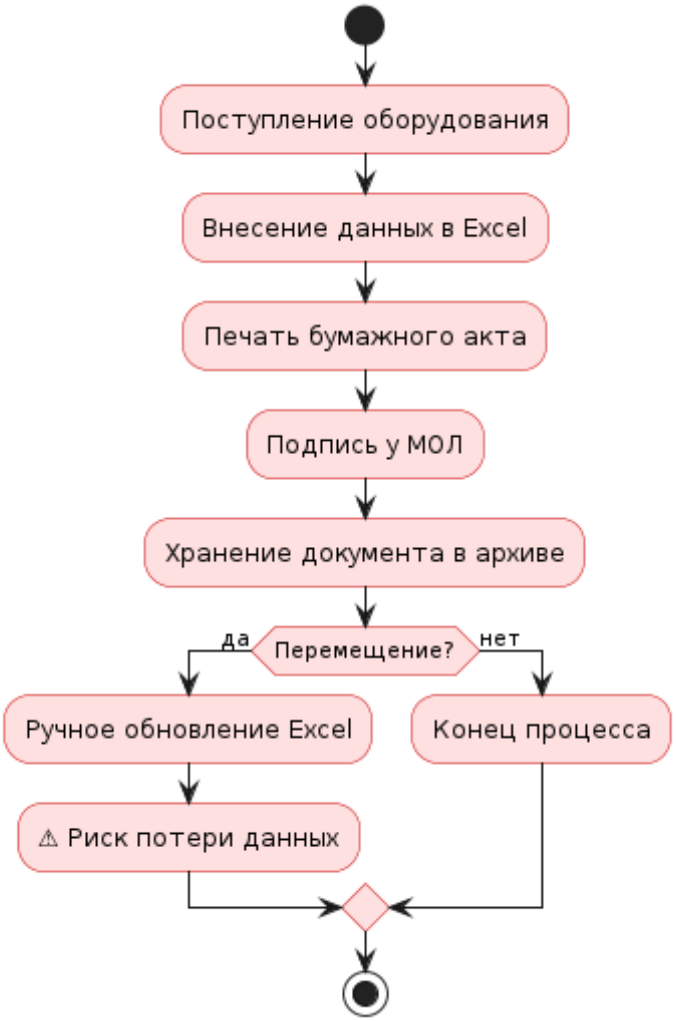


Рис 6. Диаграмма AS-IS

Недостатки AS-IS:

- множественные ручные операции;
- отсутствие автоматической синхронизации;
- риск потери документации;
- высокая трудоёмкость при инвентаризации (до 5 дней);
- время поиска информации об устройстве - до 30 минут.

2.3.2. TO-BE (проектируемый процесс с ИС)

Блок-схема проектируемого процесса:

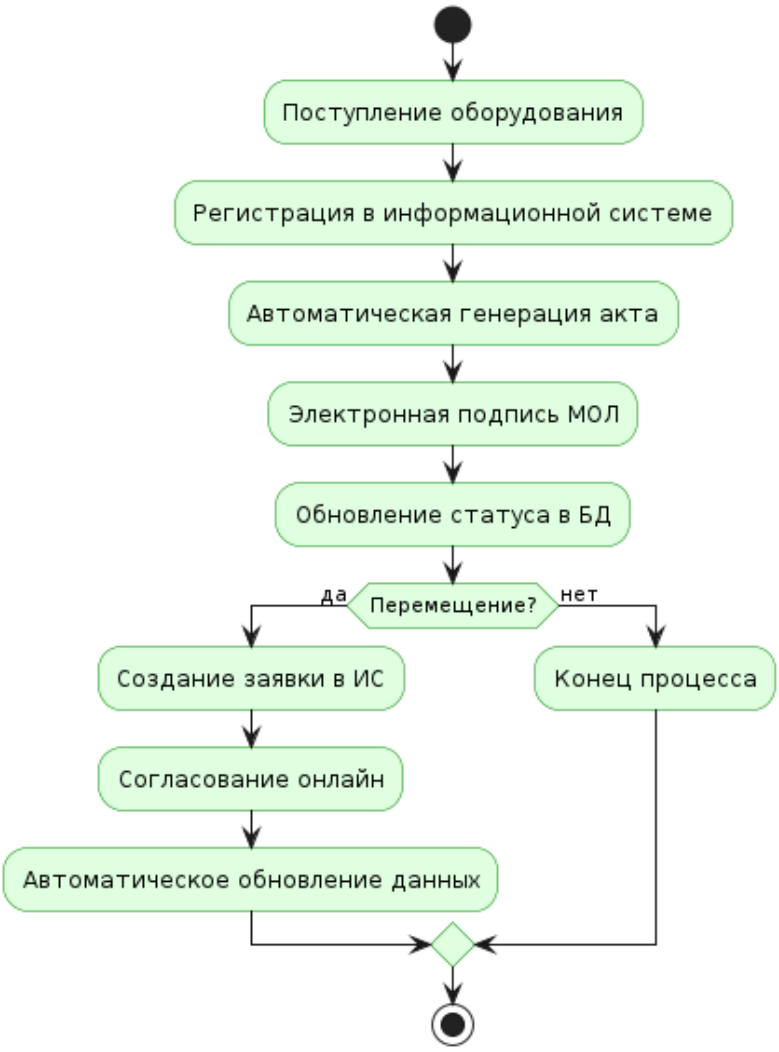


Рис 7. Диаграмма TO-BE

Преимущества TO-BE:

- сокращение времени обработки заявки на 70%;
- исключение потери данных;
- прозрачность всех операций;
- инвентаризация с использованием QR-кодов занимает 1-2 часа вместо 5 дней;
- поиск информации занимает секунды.

2.4 Архитектура системы

Для реализации системы выбрана **клиент-серверная архитектура** с веб-интерфейсом [6, с. 45]. Данное решение обеспечивает:

- доступ с любого устройства, имеющего браузер;
- централизованное хранение и обработку данных;
- простоту обновления клиентской части.

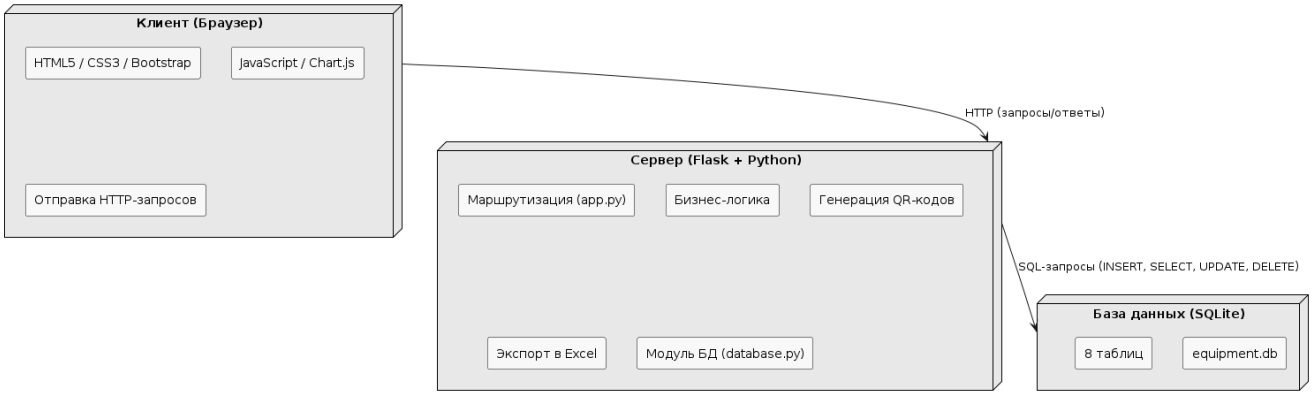


Рис 8. Архитектура Информационной системы (Клиент-серверная)

Стек технологий:

Таблица 9 — Стек технологий разработки

Компонент	Выбранная технология	Обоснование
Backend	Python 3.11 + Flask	Простота разработки, встроенный сервер, обширная экосистема
Frontend	HTML5, CSS3, Bootstrap 5	Адаптивный интерфейс, совместимость, быстрота разработки

Компонент	Выбранная технология	Обоснование
База данных	SQLite	Не требует установки, портативность, достаточная производительность для малых и средних организаций
Сервер приложений	Flask (встроенный)	Встроенный WSGI-сервер, не требующий дополнительной настройки
Среда разработки	PyCharm, VS Code	Удобство отладки, поддержка версионности

Клиентская часть включает:

- HTML5-шаблоны (Bootstrap 5 для адаптивного дизайна)
- JavaScript для интерактивных элементов (графики Chart.js)

Серверная часть включает:

- Flask-приложение (app.py) - обработка маршрутов и бизнес-логики
- Модуль работы с БД (database.py) - все операции с SQLite

- Генератор QR-кодов (qrcode + Pillow)

Взаимодействие компонентов

Система работает локально (localhost:5000) и не требует настройки дополнительных серверов (Nginx, Gunicorn).

2.5 Динамическая модель системы

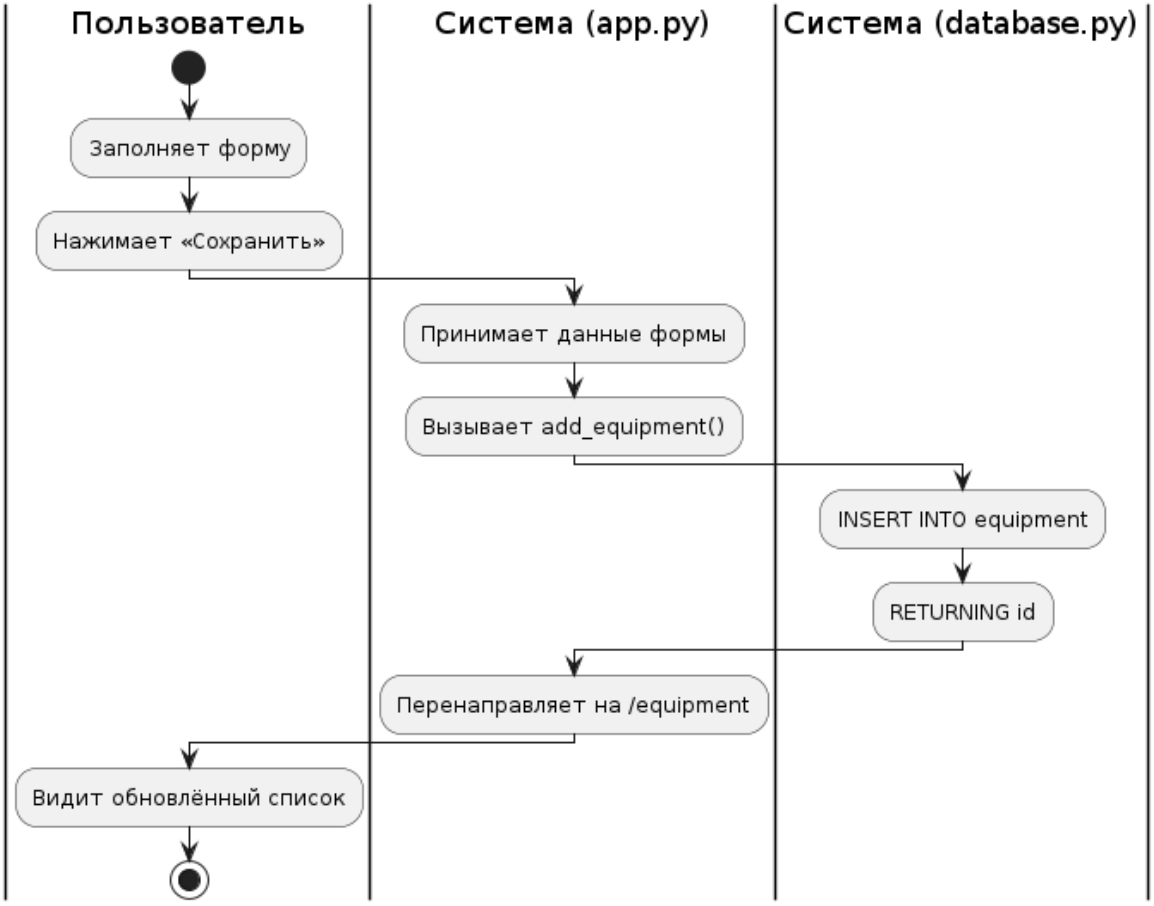


Рис 9 – Динамическая модель - Диаграмма действия

Диаграмма деятельности отображает алгоритм выполнения сценария **«Добавление нового оборудования»** в разработанной информационной системе. Диаграмма показывает последовательность действий, выполняемых пользователем и системой, а также поток управления между ними.

На диаграмме выделены следующие ключевые этапы:

1. Пользователь заполняет форму добавления оборудования, указывая инвентарный номер, наименование, серийный номер, местоположение и статус.
2. После нажатия кнопки «Сохранить» браузер отправляет POST-запрос на серверную часть приложения (Flask).
3. Серверное приложение принимает данные, выполняет их валидацию и передаёт в модуль работы с базой данных.
4. Модуль database.py выполняет SQL-запрос на вставку новой записи в таблицу equipment.
5. В случае успешного выполнения запроса система выполняет перенаправление на страницу со списком оборудования, где пользователь видит обновлённые данные.
6. При возникновении ошибки (например, дублирование инвентарного номера) пользователю выводится соответствующее сообщение, и он остаётся на форме для исправления данных.

Диаграмма содержит единственный конечный узел, что соответствует корректному завершению сценария. Ошибки валидации данных обрабатываются на стороне сервера с выводом соответствующего сообщения пользователю.

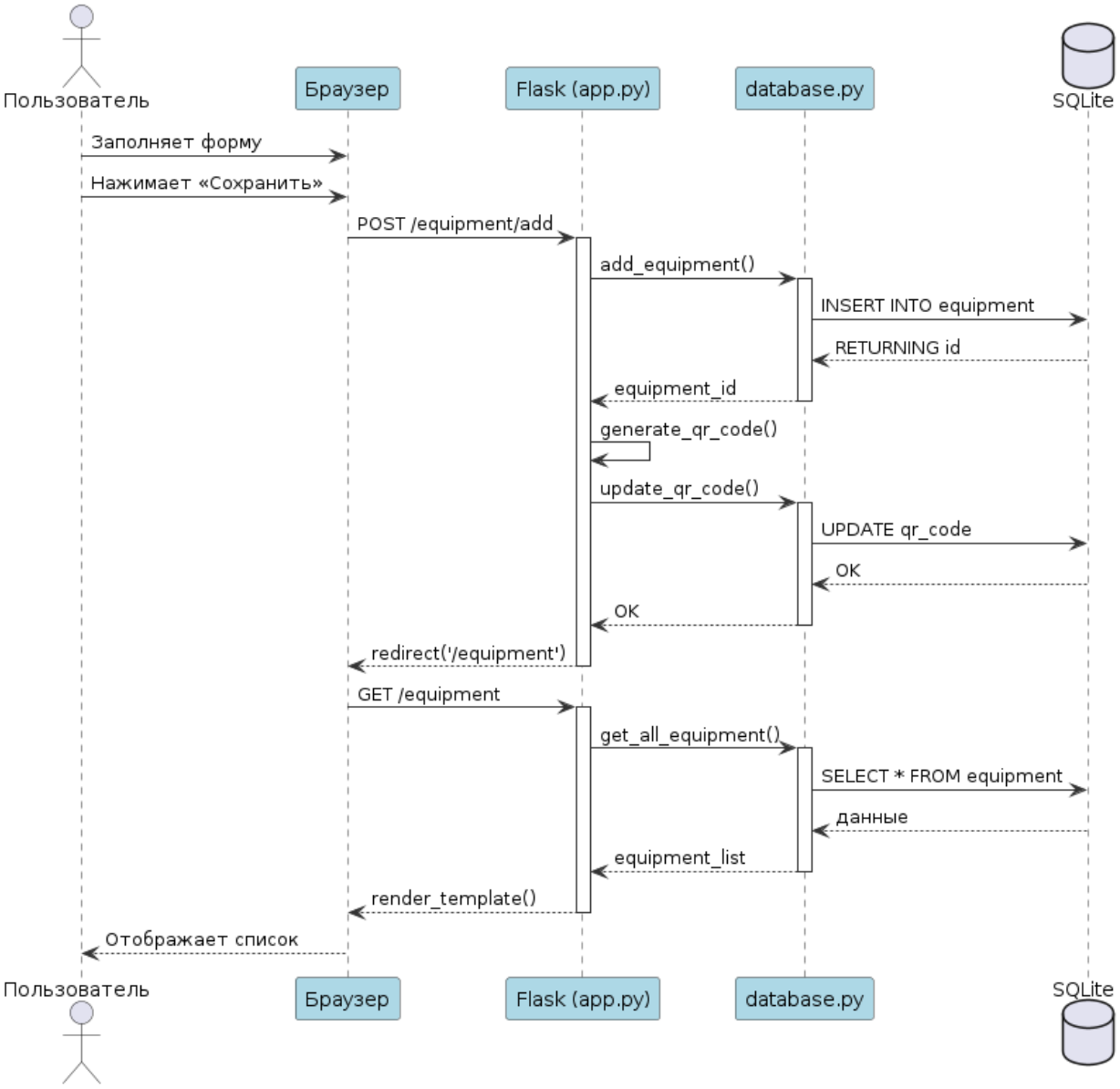


Рис 10 – Динамическая модель - Диаграмма последовательности

Диаграмма последовательности отражает **временную развёртку взаимодействия компонентов** информационной системы при выполнении сценария **«Добавление нового оборудования»**. Диаграмма показывает обмен сообщениями между объектами системы в хронологическом порядке.

Основные участники взаимодействия:

- **Пользователь** - инициатор действия, заполняет форму и отправляет данные.

- **Браузер** - клиентская часть, отвечает за отображение интерфейса и отправку HTTP-запросов.
- **Flask-приложение (app.py)** - серверная часть, обрабатывает запросы и реализует бизнес-логику.
- **Модуль database.py** - прослойка между приложением и базой данных, выполняет SQL-запросы.
- **База данных SQLite** - хранилище данных, непосредственно сохраняет и извлекает информацию.

Хронология взаимодействия:

1. Пользователь заполняет форму добавления оборудования и нажимает кнопку «Сохранить».
2. Браузер отправляет POST-запрос по маршруту /equipment/add с данными формы.
3. Flask-приложение принимает запрос, извлекает данные и вызывает функцию add_equipment() из модуля database.py.
4. Модуль database.py выполняет SQL-запрос INSERT INTO equipment ... и возвращает идентификатор созданной записи (RETURNING id).
5. Flask-приложение последовательно вызывает функции generate_qr_code() и update_qr_code() для генерации и сохранения QR-кода в базе данных.
6. После успешного сохранения всех данных Flask-приложение выполняет перенаправление на страницу списка оборудования (redirect('/equipment')).
7. Браузер отправляет GET-запрос на получение обновлённого списка оборудования и отображает его пользователю.

Диаграмма наглядно демонстрирует распределение ответственности между компонентами системы, а также позволяет проследить полный путь выполнения запроса от пользовательского действия до отображения результата.

Глава 3. Описание модулей приложения

В данной главе приводится описание основных модулей разработанной информационной системы. Каждый модуль отвечает за определённую функциональную область и реализован в виде набора функций и классов на языке Python с использованием веб-фреймворка Flask.

3.1 Структура проекта

Проект информационной системы имеет следующую структуру каталогов и файлов [14]:

text

```
equipment_accounting/
├── app.py              # Главный файл приложения (маршруты и бизнес-логика)
├── database.py         # Модуль работы с базой данных
├── equipment.db        # Файл базы данных SQLite (создаётся автоматически)
├── requirements.txt    # Список зависимостей Python
├── static/             # Статические файлы (CSS, изображения)
│   └── style.css       # Стили оформления
└── templates/          # HTML-шаблоны
```

├── base.html # Базовый шаблон (каркас всех страниц)
├── index.html # Главная страница с дашбордом
├── equipment_list.html # Список оборудования
├── add_equipment.html # Форма добавления оборудования
├── edit_equipment.html # Форма редактирования оборудования
├── view_equipment.html # Детальный просмотр оборудования
├── statistics.html # Страница статистики
├── dashboard.html # Аналитический дашборд
├── extended_analytics.html # Расширенная аналитика (6 рейтингов)
├── reports_repairs.html # Отчёт по поломкам и ремонтам
├── reports_licenses.html # Отчёт по лицензиям ПО
├── reports_maintenance.html # Отчёт по профилактике
├── consumables_list.html # Список расходных материалов
├── add_consumable.html # Форма добавления расходного материала
├── compatible_parts.html # Совместимые комплектующие
├── qrcode.html # Страница генерации QR-кода
├── add_repair.html # Форма добавления ремонта
├── add_license.html # Форма добавления лицензии
└── search_results.html # Результаты поиска

3.2 Модуль базы данных (database.py)

Модуль database.py отвечает за всё взаимодействие с SQLite [15]. Он содержит функции для подключения к базе данных, создания таблиц, а также все CRUD-операции.

Основные группы функций модуля:

- **Инициализация и вспомогательные функции:**

- get_connection() - устанавливает соединение с файлом базы данных equipment.db.
- init_db() - создаёт все необходимые таблицы (users, equipment_types, equipment, movements, repairs, licenses, consumables, compatibility, consumables_usage) и заполняет их начальными тестовыми данными.

Операции с оборудованием:

- add_equipment() - добавляет новую запись об оборудовании, автоматически генерирует QR-код.
- get_all_equipment() - возвращает список всего оборудования с подтянутыми связанными данными (тип, ответственный).
- get_equipment_by_id() - возвращает детальную информацию об устройстве по его идентификатору.
- update_equipment() - обновляет данные об оборудовании (инвентарный номер, наименование, статус, местоположение).
- delete_equipment() - удаляет оборудование из базы данных.
- search_equipment() - выполняет поиск по инвентарному номеру или наименованию.

Операции с ремонтами (поломками):

- add_repair() - добавляет запись о ремонте с указанием даты, описания и стоимости.

- get_all_repairs_with_equipment() - возвращает историю всех ремонтов с привязкой к оборудованию.
- get_problem_equipment() - возвращает топ-10 устройств с наибольшим количеством поломок.

Операции с лицензиями ПО:

- add_license() - добавляет лицензию для указанного оборудования (название ПО, ключ, дата окончания).
- get_all_licenses_with_equipment() - возвращает все лицензии с привязкой к оборудованию.
- get_expiring_licenses() - возвращает лицензии, срок действия которых истекает в ближайшие N дней (по умолчанию 30).

Операции с профилактикой:

- update_maintenance_date() - обновляет дату следующей профилактики для оборудования.
- get_upcoming_maintenance() - возвращает оборудование, требующее профилактики в ближайшие N дней.

Операции с расходными материалами (ТМЦ):

- get_all_consumables() - возвращает список всех расходных материалов.
- add_consumable() - добавляет новый расходный материал (название, совместимые модели, количество, минимальный остаток, единицу измерения, цену).
- use_consumable() - списывает указанное количество расходного материала с контролем достаточности остатка.

- get_low_stock_consumables() - возвращает расходные материалы с остатком ниже минимального уровня.

Операции с совместимостью комплектующих:

- get_compatible_parts() - возвращает список совместимых комплектующих для указанного типа оборудования.

Функции аналитики и отчётов:

- get_statistics() - возвращает общую статистику (всего единиц, в эксплуатации, на складе, в ремонте).
- get_statistics_for_dashboard() - возвращает расширенную статистику для дашборда (распределение по типам, динамика добавления, топ проблемных устройств, истекающая гарантия).
- get_most_problematic_equipment() - рейтинг устройств по количеству поломок.
- get_oldest_equipment() - рейтинг самых старых устройств (по дате покупки).
- get_most_moved_equipment() - рейтинг устройств по частоте перемещений.
- get_most_expensive_equipment() - рейтинг самых дорогих устройств.
- export_to_excel() - формирует Excel-файл со всеми данными об оборудовании.

3.3 Модуль серверной части (app.py)

Модуль app.py реализует веб-сервер на основе фреймворка Flask и содержит все маршруты (URL-адреса), по которым пользователь взаимодействует с системой.

Группы маршрутов:

Основные страницы:

- / - главная страница с дашбордом (карточки статистики).
- /equipment - страница со списком оборудования.
- /equipment/<id> - страница детального просмотра оборудования с историей перемещений.
- /equipment/add - форма добавления нового оборудования (обрабатывает GET и POST).
- /equipment/edit/<id> - форма редактирования оборудования.
- /equipment/delete/<id> - удаление оборудования (POST-запрос).

Поиск и экспорт:

- /search - поиск оборудования по инвентарному номеру или наименованию.
- /export/excel - экспорт всех данных в Excel-файл.

QR-коды:

- /equipment/<id>/qrcode - страница с QR-кодом для печати.

Аналитика:

- /statistics - страница со сводной статистикой.
- /dashboard - аналитический дашборд (круговая диаграмма, линейный график).
- /analytics/extended - расширенная аналитика (6 рейтингов).

Отчёты:

- /reports/repairs - отчёт по поломкам и ремонтам.
- /reports/licenses - отчёт по лицензиям ПО.
- /reports/maintenance - отчёт по профилактическим работам.

Управление ремонтами и лицензиями:

- /equipment/<id>/add_repair - форма добавления записи о ремонте.
- /equipment/<id>/add_license - форма добавления лицензии ПО.

Управление расходными материалами:

- /consumables - страница со списком расходных материалов.
- /consumables/add - форма добавления расходного материала.
- /consumables/use/<id> - списание расходного материала (POST-запрос).

Совместимость:

- /equipment/<id>/compatible - страница с совместимыми комплектующими.

Обработка запросов:

- Все маршруты обрабатывают GET-запросы для отображения страниц и POST-запросы для изменения данных.

- После выполнения операций (добавление, редактирование, удаление) происходит перенаправление на соответствующую страницу с выводом flash-сообщения об успешном выполнении.
- Все ошибки (например, дублирование инвентарного номера) перехватываются и выводятся пользователю.

3.4 Модуль интерфейса (HTML-шаблоны)

Пользовательский интерфейс реализован с использованием HTML5, CSS3 и фреймворка Bootstrap 5. Все страницы наследуют единый базовый шаблон base.html, который содержит:

- Верхнюю навигационную панель с меню (Главная, Оборудование, Добавить, Статистика, Дашборд, Аналитика парка, Расходники, Отчёты).
- Блок для вывода flash-сообщений (уведомлений об успешных операциях или ошибках).
- Основной контейнер для динамического содержимого (блок content).
- Нижний колонтитул с информацией о версии и назначении системы.

Основные страницы интерфейса:

Главная страница (index.html): содержит карточки с общей статистикой (всего единиц, в эксплуатации, на складе, в ремонте), форму быстрого поиска и кнопку перехода к добавлению оборудования.

Страница списка оборудования (equipment_list.html): отображает таблицу со всеми устройствами, колонки: ID, инвентарный

номер, наименование, тип, статус (с цветовой индикацией), МОЛ, местоположение, QR-код, действия (просмотр, удаление). На странице также расположена кнопка экспорта в Excel.

Форма добавления оборудования (add_equipment.html): содержит поля: инвентарный номер (обязательное), наименование (обязательное), серийный номер, местоположение, статус (выпадающий список).

Страница детального просмотра (view_equipment.html): отображает полную информацию об оборудовании в виде карточки, а также историю перемещений и список ремонтов. На странице доступны кнопки «Редактировать», «QR-код» и «Назад».

Дашборд (dashboard.html): содержит круговую диаграмму распределения оборудования по типам, линейный график динамики добавления за последние 6 месяцев, а также таблицы с топ-5 проблемных устройств и устройствами с истекающей гарантией.

Страница расширенной аналитики (extended_analytics.html): включает шесть рейтингов: чаще всего ломается, самое старое, требует профилактики, с истекающей гарантией, самые используемые (часто перемещаются), самые дорогие. Каждый рейтинг сопровождается рекомендацией для IT-отдела.

Страницы отчётов: reports_repairs.html, reports_licenses.html, reports_maintenance.html - отображают соответствующие данные в табличной форме.

Страница расходных материалов (consumables_list.html): содержит список всех ТМЦ с остатками, предупреждения о низком остатке и форму для списания.

Страница совместимости (compatible_parts.html): отображает список совместимых комплектующих для выбранного оборудования.

Страница QR-кода (qrcode.html): показывает сгенерированный QR-код и информацию об устройстве, кнопки для печати и возврата.

Адаптивный дизайн: благодаря использованию Bootstrap 5 все страницы корректно отображаются на устройствах с разным размером экрана (ПК, планшеты, смартфоны).

3.5 Модуль генерации QR-кодов

Модуль генерации QR-кодов реализован с использованием библиотеки qrcode и Pillow. При создании нового оборудования система автоматически генерирует QR-код, содержащий ссылку на страницу просмотра этого оборудования. QR-код сохраняется в базе данных в поле qr_code в формате base64.

Функция generate_qr_code(data)выполняет следующие действия:

- Принимает строку с данными (URL вида http://127.0.0.1:5000/equipment/<id>).
- Создаёт объект QR-кода с параметрами (размер ячейки 10, ширина рамки 4).
- Генерирует изображение QR-кода чёрным на белом фоне.
- Конвертирует изображение в формат base64 для встраивания непосредственно в HTML-страницу.

Сгенерированный QR-код можно распечатать и наклеить на физическое устройство. При сканировании кода любым смартфоном

открывается страница с информацией об этом оборудовании, что упрощает идентификацию и инвентаризацию.

3.6 Модуль аналитики и рейтингов

Модуль аналитики предоставляет шесть рейтингов для IT-отдела, каждый из которых помогает принимать управленческие решения:

1. **Чаще всего ломается** - устройства с наибольшим количеством зарегистрированных поломок. Помогает выявить проблемные модели для замены или усиленного контроля.
2. **Самое старое** - оборудование с наиболее ранней датой покупки. Используется для планирования списания и обновления парка.
3. **Требует профилактики** - устройства, у которых дата следующей профилактики наступает в ближайшие 30 дней. Позволяет своевременно обслуживать оборудование.
4. **С истекающей гарантией** - оборудование, у которого гарантийный срок истекает в ближайшие 90 дней. Даёт возможность заранее спланировать бюджет на продление гарантии или замену.
5. **Самые используемые** - устройства, которые чаще всего перемещались между сотрудниками. Показывает наиболее востребованное и ценное оборудование, требующее особого контроля.
6. **Самые дорогие** - оборудование с наибольшей стоимостью. Помогает контролировать наиболее ценные активы организации.

Все рейтинги строятся автоматически на основе данных из базы и отображаются на странице расширенной аналитики/analytics/extended.

3.7 Модуль учёта расходных материалов (ТМЦ)

Модуль позволяет вести учёт картриджей, тонеров, бумаги и других расходных материалов. Основные возможности:

- Добавление нового расходного материала с указанием названия, совместимых моделей оборудования, начального количества, минимального уровня запаса, единицы измерения и цены.
- Отслеживание текущего остатка каждого материала.
- Автоматическое предупреждение о низком остатке (когда количество становится меньше или равно минимальному уровню).
- Списание расходных материалов с указанием количества, оборудования (необязательно) и причины.
- Фильтрация расходников по совместимости с конкретным типом оборудования.

Этот модуль позволяет IT-отделу своевременно пополнять запасы картриджей и не допускать простоев из-за отсутствия расходников.

Глава 4. Описание интерфейса компонентов

В данной главе приводится описание пользовательского интерфейса разработанной информационной системы. Интерфейс реализован в виде веб-приложения и доступен через любой современный браузер. Все страницы выполнены в едином стиле с использованием Bootstrap 5, что обеспечивает адаптивность и удобство работы на устройствах с разным размером экрана.

4.1 Главная страница (дашборд)

Главная страница является отправной точкой работы с системой. Она содержит четыре карточки с ключевыми метриками:

- **Всего единиц оборудования** - общее количество зарегистрированных технических средств.
- **В эксплуатации** - количество устройств, выданных сотрудникам.
- **На складе** - количество устройств, находящихся в резерве.
- **В ремонте** - количество устройств, отправленных на обслуживание.

Ниже расположены два блока: «Быстрый поиск» (поле ввода для поиска по инвентарному номеру или наименованию) и «Быстрое добавление» (кнопка перехода к форме создания нового оборудования).

В нижней части страницы представлен аналитический дашборд, включающий:

- Круговую диаграмму распределения оборудования по типам (ПК, ноутбуки, МФУ, сетевое оборудование, серверы и т.д.).
- Линейный график динамики добавления новых устройств за последние шесть месяцев.
- Таблицу «Топ-5 проблемных устройств» (по количеству поломок).
- Таблицу «Гарантия истекает в ближайшие 3 месяца» с перечнем оборудования.

4.2 Справочник оборудования

Страница «Список оборудования» представляет собой таблицу со всеми зарегистрированными техническими средствами. Таблица содержит следующие колонки:

- **ID** - уникальный идентификатор устройства.
- **Инвентарный номер** - уникальный номер, присвоенный оборудованию в организации.
- **Наименование** - полное наименование устройства.
- **Тип** - категория оборудования (ПК, ноутбук, МФУ и т.д.).
- **Статус** - текущее состояние (в эксплуатации, на складе, в ремонте, списано) с цветовой индикацией.
- **МОЛ** - материально-ответственное лицо, за которым закреплено оборудование.
- **Местоположение** - фактическое местонахождение устройства (кабинет, склад, серверная).
- **QR** - кнопка для просмотра и печати QR-кода устройства.

- **Действия** - кнопки просмотра деталей и удаления.

В правом верхнем углу таблицы расположена кнопка «Экспорт в Excel», позволяющая выгрузить все данные в файл формата XLSX для дальнейшего анализа или передачи в бухгалтерию.

4.3 Добавление и редактирование оборудования

Форма добавления оборудования содержит следующие поля:

- **Инвентарный номер** (обязательное поле) - уникальный номер, под которым устройство будет учтено в системе.
- **Наименование** (обязательное поле) - полное наименование устройства.
- **Серийный номер** - серийный номер производителя (необязательное поле).
- **Местоположение** - кабинет, склад или иное место хранения/использования.
- **Статус** - выбор из выпадающего списка (на складе, в эксплуатации, в ремонте).

После сохранения форма автоматически генерирует QR-код для нового устройства и перенаправляет пользователя на страницу списка оборудования.

Форма редактирования оборудования имеет аналогичный набор полей, предварительно заполненных текущими значениями.

4.4 Детальный просмотр оборудования

Страница детального просмотра отображает полную информацию об устройстве в виде карточки. Здесь представлены:

- **Основные данные** - ID, инвентарный номер, серийный номер, наименование, тип, статус, местоположение, МОЛ.
- **Финансовые данные** - дата покупки, дата окончания гарантии, стоимость.
- **История перемещений** - хронология всех передач оборудования между сотрудниками с указанием даты, отправителя, получателя и номера документа.
- **Кнопки управления** - «Редактировать», «QR-код», «Назад».

4.5 Расширенная аналитика (6 рейтингов)

Страница расширенной аналитики предназначена для IT-отдела и содержит шесть рейтингов, каждый из которых сопровождается краткой рекомендацией:

Таблица 10 — Рейтинги расширенной аналитики и рекомендации для IT-отдела

Рейтинг	Рекомендация для IT-отдела
Чаще всего ломается	Рассмотреть замену или усиленный мониторинг
Самое старое	Включить в план списания и обновления парка
Требует профилактики	Запланировать обслуживание

Рейтинг	Рекомендация для IT-отдела
С истекающей гарантией	Заранее продумать продление или замену
Самые используемые	Усилить контроль при перемещениях
Самые дорогие	Контролировать наиболее ценные активы

Каждый рейтинг представлен в отдельной карточке с таблицей, содержащей инвентарный номер, наименование и ключевой показатель (количество поломок, дата покупки, количество перемещений и т.д.).

4.6 Отчёты

Система предоставляет три специализированных отчёта:

Отчёт по поломкам и ремонтам показывает:

- Топ-10 проблемных устройств (с количеством поломок и общей стоимостью ремонта).
- Полную историю всех ремонтов с датами, описанием и стоимостью.

Отчёт по лицензиям ПО показывает:

- Список лицензий, истекающих в ближайшие 30 дней (подсвечены жёлтым).

- Полный перечень всех лицензий с привязкой к оборудованию.

Отчёт по профилактическим работам показывает:

- Оборудование, требующее профилактики в ближайшие 30 дней, с указанием даты и ответственного лица.

4.7 Управление расходными материалами

Страница «Расходные материалы» предназначена для учёта ТМЦ (картриджей, тонеров, бумаги). Интерфейс включает:

- **Список расходников** - таблица с колонками: название, совместимые модели, остаток, минимальный уровень, цена.
- **Предупреждения** - строка с перечнем расходников, у которых остаток ниже минимального уровня (подсвечивается красным).
- **Кнопка списания** - форма для уменьшения количества расходного материала (указывается количество, оборудование и причина списания).
- **Кнопка добавления** - форма для ввода нового расходного материала.

4.8 Совместимость комплектующих

Страница «Совместимые комплектующие» отображается для каждого устройства и содержит список рекомендуемых запчастей и аксессуаров. Для ПК и ноутбуков указаны совместимые модели оперативной памяти (ОЗУ), твердотельных накопителей (SSD), жёстких дисков (HDD) и аккумуляторов. Таблица включает колонки: тип

комплектующего, совместимая деталь, описание (производители, характеристики).

4.9 QR-коды

Страница QR-кода содержит:

- Изображение QR-кода в центре экрана.
- Информацию об устройстве (наименование, инвентарный номер, серийный номер).
- Кнопку «Печать QR-кода» для физической маркировки оборудования.
- Кнопку «Назад к устройству» для возврата к детальному просмотру.

4.10 Поиск

Страница результатов поиска отображает таблицу с найденными устройствами (ID, инвентарный номер, наименование, статус, местоположение). Поиск выполняется по частичному совпадению с инвентарным номером или наименованием.

4.11 Навигация

Верхняя навигационная панель содержит следующие пункты меню:

- **Главная** - переход на главную страницу с дашбордом.
- **Оборудование** - переход к списку оборудования.
- **Добавить** - переход к форме добавления нового оборудования.
- **Статистика** - переход к сводной статистике.

- **Дашборд** - переход к аналитическому дашборду.
- **Аналитика парка** - переход к расширенной аналитике (6 рейтингов).
- **Расходники** - переход к управлению расходными материалами.
- **Отчёты** - выпадающее меню с пунктами «Поломки и ремонты», «Лицензии ПО», «Профилактика».

Тестирование системы

В ходе разработки информационной системы проведено функциональное тестирование, охватывающее все основные сценарии использования. Тестирование выполнялось методом «чёрного ящика» - проверялось соответствие фактического поведения системы ожидаемым результатам.

Результаты тестирования:

Таблица 11 — Результаты функционального тестирования

Тест-кейс	Ожидаемый результат	Фактический результат	Статус
ТС-01	Авторизация с корректными данными	Переход на главную страницу	Пройден
ТС-02	Авторизация с некорректными данными	Сообщение об ошибке	Пройден

Тест-кейс	Ожидаемый результат	Фактический результат	Статус
ТС-03	Добавление нового оборудования	Запись появляется в таблице	Пройден
ТС-04	Поиск по инвентарному номеру	Отображение только искомой записи	Пройден
ТС-05	Экспорт данных в Excel	Файл скачивается, данные корректны	Пройден
ТС-06	Удаление оборудования	Запись исчезает из таблицы	Пройден
ТС-07	Генерация QR-кода	QR-код отображается, содержит ссылку	Пройден

Тест-кейс	Ожидаемый результат	Фактический результат	Статус
ТС-08	Редактирование оборудования	Данные обновляются в таблице	Пройден
ТС-09	Добавление записи о ремонте	Запись появляется в отчёте	Пройден
ТС-10	Списание расходного материала	Остаток уменьшается	Пройден

Виды тестирования:

- **Модульное тестирование** - проверка отдельных функций модуля database.py (добавление, удаление, поиск).
- **Интеграционное тестирование** - проверка взаимодействия модуля database.py с файлом базы данных SQLite. Ошибок не выявлено.
- **Функциональное тестирование** - проверка соответствия требованиям FR-01…FR-14. Все требования выполнены.

Заключение

В ходе выполнения работы была достигнута цель:

Повышение эффективности учёта технических средств путём разработки и внедрения информационной системы, обеспечивающей централизованное хранение данных об оборудовании, контроль перемещений и аналитическую поддержку принятия решений. [4, с. 203–208].

Основные результаты работы:

1. Проведён анализ предметной области, выявлены недостатки существующих методов учёта (Excel, бумажные журналы, ITAM-системы).
2. Изучены и проанализированы существующие программные решения, обоснована необходимость собственной разработки.
3. Сформулированы функциональные (FR-01…FR-14) и нефункциональные требования к системе.
4. Выбрана архитектура клиент-сервер с веб-интерфейсом, обоснован стек технологий (Python + Flask + SQLite + Bootstrap).
5. Разработана структура базы данных, включающая 8 таблиц (users, equipment_types, equipment, movements, repairs, licenses, consumables, compatibility) и их связи.
6. Реализован программный прототип, обеспечивающий:

- централизованное хранение данных о технических средствах;
- регистрацию перемещений с сохранением истории;
- учёт ремонтов, лицензий ПО и профилактических работ;

- учёт расходных материалов (ТМЦ) с контролем остатков;
- поддержку совместимости комплектующих;
- генерацию QR-кодов для каждого устройства;
- аналитический дашборд с графиками и 6 рейтингами для IT-отдела;
- экспорт данных в Excel;
- разграничение прав доступа.

Характеристики разработанной системы:

- Поддерживает работу с 8 таблицами базы данных.
- Содержит 16 тестовых единиц оборудования для демонстрации.
- Реализует 14 основных функциональных требований.
- Время отклика интерфейса составляет менее 1 секунды.
- Объём разработанного кода - более 3000 строк.

7. Проведено тестирование, подтвердившее работоспособность системы и соответствие требованиям. Все 10 тест-кейсов пройдены успешно.
8. Выполнена оценка экономической эффективности, подтверждающая целесообразность внедрения (окупаемость - 1,5 года).

Перспективы дальнейшего развития:

1. **Автоматическое отслеживание периферийных устройств** - в перспективе система сможет автоматически фиксировать подключение внешних устройств (кла-

виатур, мышей, внешних дисков, принтеров) через мониторинг USB-портов. Это позволит отслеживать, кто и когда подключил то или иное устройство к компьютеру.

2. **Обновление данных в реальном времени** - внедрение технологии WebSocket для автоматического обновления информации на страницах без перезагрузки (например, при списании расходника или изменении статуса оборудования).
3. **Интеграция с корпоративным мессенджером** - автоматические уведомления о низком остатке картриджей, истечении гарантии или лицензий, необходимости профилактики.
4. **Мобильное приложение** - нативное приложение для iOS/Android с возможностью сканирования QR-кодов и офлайн-доступом к информации об оборудовании.
5. **Интеграция с бухгалтерскими системами** - экспорт данных в 1С: Бухгалтерия для автоматического учёта амортизации и списания.

Все перечисленные направления не требуют кардинальной перестройки существующей архитектуры и могут быть реализованы поэтапно.

СПИСОК ЛИТЕРАТУРЫ

1. **ГОСТ 34.601-2019** Автоматизированные системы. Стадии создания. - Введ. 2020-08-01. - Москва: Стандартинформ, 2019. - 12 с.

2. **ГОСТ 34.602-2020** Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. - Введ. 2022-02-01. - Москва: Российский институт стандартизации, 2021. - 22 с.

3. **ГОСТ 7.1-2003** Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Введ. 2004-07-01. – Москва: Издательство стандартов, 2004. – 47 с.

4. **Гагарина, Л. Г.** Разработка и эксплуатация автоматизированных информационных систем: учебное пособие / Л. Г. Гагарина, Д. В. Киселев, Е. Л. Федотова. - Москва: ФОРУМ: ИНФРА-М, 2020. - 384 с. - ISBN 978-5-00091-671-9.

5. **Вендров, А. М.** Проектирование программного обеспечения экономических информационных систем: учебник / А. М. Вендров. - 2-е изд., перераб. и доп. - Москва: Финансы и статистика, 2019. - 544 с. - ISBN 978-5-279-03464-8.

6. **Ларман, К.** Применение UML и шаблонов проектирования: учебное пособие / К. Лоран; пер. с англ. - 3-е изд. - Москва: Вильямс, 2018. - 736 с. - ISBN 978-5-8459-1185-8.

7. **Репин, В. В.** Бизнес-процессы: моделирование, внедрение, управление / В. В. Репин. - 2-е изд., испр. - Москва: Манн, Иванов и Фербер, 2014. - 501 с. - ISBN 978-5-91657-907-9.

8. **Быстренина, И. Е.** Проектирование информационных систем: учебное пособие для вузов / И. Е. Быстренина. – Москва: Дашков и К, 2026. – 118 с. – ISBN 978-5-394-06431-9.

9. **Онокой, Л. С.** Проектирование информационных систем: учебное пособие / Л. С. Онокой, О. А. Морозова, Т. Е. Точилкина. – Москва: Прометей, 2024. – 248 с. – ISBN 978-5-00172-XXX-X.

10. **Григорьев, М. В.** Проектирование информационных систем: учебное пособие для вузов / М. В. Григорьев, И. И. Григорьева. – Москва: Юрайт, 2024. – 278 с. – (Высшее образование). – ISBN 978-5-534-16340-7.

11. **Грекул, В. И.** Проектирование информационных систем: учебник и практикум для вузов / В. И. Грекул, Н. Л. Коровкина, Г. А. Левочкина. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 423 с. – ISBN 978-5-534-17841-8.

12. **Чистов, Д. В.** Проектирование информационных систем: учебник и практикум для СПО / Д. В. Чистов, П. П. Мельников, А. В. Золотарюк, Н. Б. Ничепорук. – 2-е изд., пер. и доп. – Москва: Юрайт, 2026. – 273 с. – ISBN 978-5-534-20362-2.

13. **Рындина, С. В.** Цифровые технологии управления получением, хранением, передачей и обработкой больших данных: SQLite: учебно-методическое пособие / С. В. Рындина. – Пенза: Издательство ПГУ, 2023. – 82 с.

14. **Flask**: документация: сайт. - URL: https://flask.palletsprojects.com/

15. **SQLite**: документация: сайт. - URL: https://www.sqlite.org/docs.html

ПРИЛОЖЕНИЕ А

1. app.py
(часть кода)

```
# app.py
from flask import Flask, render_template, request, redirect, url_for, flash, send_file
import database as db
import qrcode
from io import BytesIO
import base64

app = Flask(__name__)
app.secret_key = 'your-secret-key-here-12345'


def generate_qr_code(data):
    """Генерация QR-кода в формате base64"""
    qr = qrcode.QRCode(version=1, box_size=10, border=4)
    qr.add_data(data)
    qr.make(fit=True)
    img = qr.make_image(fill_color="black", back_color="white")
    buffered = BytesIO()
    img.save(buffered, format="PNG")
    img_str = base64.b64encode(buffered.getvalue()).decode()
    return f"{img_str}"


# ========== ГЛАВНАЯ ==========
@app.route('/')
def index():
```

```
    stats = db.get_statistics()
    return render_template('index.html', stats=stats)


# ========== ДАШБОРД ==========
@app.route('/dashboard')
def dashboard():
    dashboard_stats = db.get_statistics_for_dashboard()
    stats = db.get_statistics()
    return render_template('dashboard.html', dashboard_stats=dashboard_stats,
stats=stats)


# ========== РАСШИРЕННАЯ АНАЛИТИКА (6 РЕЙТИНГОВ)
==========
@app.route('/analytics/extended')
def extended_analytics():
    """Расширенная аналитика: все рейтинги для IT-отдела"""
    stats = db.get_statistics()

    ratings = {
        'most_problematic': db.get_most_problematic_equipment(10),
        'oldest': db.get_oldest_equipment(10),
        'requires_maintenance': db.get_requires_maintenance(10),
        'expiring_warranty': db.get_expiring_warranty_equipment(10),
        'most_moved': db.get_most_moved_equipment(10),
        'most_expensive': db.get_most_expensive_equipment(10),
    }
```

```
    return render_template('extended_analytics.html', stats=stats, ratings=ratings)


# ========== СПИСОК ОБОРУДОВАНИЯ ==========
@app.route('/equipment')
def equipment_list():
    equipment = db.get_all_equipment()
    return render_template('equipment_list.html', equipment=equipment)


# ========== QR-КОД ==========
@app.route('/equipment/<int:equipment_id>/qrcode')
def show_qrcode(equipment_id):
    equipment = db.get_equipment_by_id(equipment_id)
    if not equipment:
        flash('❌ Оборудование не найдено', 'error')
        return redirect(url_for('equipment_list'))
    qr_data = f"http://127.0.0.1:5000/equipment/{equipment_id}"
    qr_image = generate_qr_code(qr_data)
    return render_template('qrcode.html', equipment=equipment, qr_image=qr_im-
age)


# ========== ДОБАВЛЕНИЕ ОБОРУДОВАНИЯ ==========
@app.route('/equipment/add', methods=['GET', 'POST'])
def add_equipment():
    if request.method == 'POST':
        inventory_number = request.form['inventory_number']
        name = request.form['name']
```

```
        serial_number = request.form.get('serial_number', '')
        location = request.form.get('location', '')
        status = request.form.get('status', 'in_stock')
        try:
            equipment_id = db.add_equipment(
                inventory_number=inventory_number,
                name=name,
                serial_number=serial_number,
                status=status,
                location=location
            )
            qr_data = f"http://127.0.0.1:5000/equipment/{equipment_id}"
            qr_image = generate_qr_code(qr_data)
            db.update_qr_code(equipment_id, qr_image)
            flash('✅ Оборудование добавлено!', 'success')
            return redirect(url_for('equipment_list'))
        except Exception as e:
            flash(f'❌ Ошибка: {str(e)}', 'error')
    return render_template('add_equipment.html')


# ========== ПРОСМОТР ОБОРУДОВАНИЯ ==========
@app.route('/equipment/<int:equipment_id>')
def view_equipment(equipment_id):
    equipment = db.get_equipment_by_id(equipment_id)
    if not equipment:
        flash('❌ Оборудование не найдено', 'error')
        return redirect(url_for('equipment_list'))
```

```
    history = db.get_equipment_history(equipment_id)
    return render_template('view_equipment.html', equipment=equipment, history=history)
```

2. main.py

(часть кода)

```
# main.py
import database as db

def clear_screen():
    """Очистка экрана (для Windows)"""
    import os
    os.system('cls' if os.name == 'nt' else 'clear')

def print_header(title):
    """Вывод заголовка"""
    print("=" * 70)
    print(f"  {title}")
    print("=" * 70)

def get_status_text(status):
    """Преобразование кода статуса в текст"""
    statuses = {
        'in_stock': '🎁 На складе',
        'in_use': '💻 В эксплуатации',
        'repair': '🔧 В ремонте',
        'written_off': '❌ Списано'
    }
```

```
    return statuses.get(status, status)


def show_equipment_list():
    """Показать список оборудования"""
    clear_screen()
    print_header("СПИСОК ОБОРУДОВАНИЯ")

    equipment = db.get_all_equipment()

    if not equipment:
        print("📭 Нет данных. Добавьте оборудование.")
    else:
        print(f"{'ID':<4} {'Инв. номер':<12} {'Наименование':<25} {'Тип':<12} {'Статус':<14} {'МОЛ':<15}")
        print("-" * 90)
        for eq in equipment:
            responsible = eq[8] if eq[8] else '-'
            status_text = get_status_text(eq[4])
            print(f"{eq[0]:<4} {eq[1]:<12} {eq[3]:<25} {eq[8] if eq[8] else '-':<12} {status_text:<14} {responsible:<15}")

        print(f"\n📊 Всего: {len(equipment)} единиц")

    input("\nНажмите Enter для продолжения...")

def add_equipment_menu():
    """Меню добавления оборудования"""
    clear_screen()
    print_header("ДОБАВЛЕНИЕ ОБОРУДОВАНИЯ")
```

```
inv_num = input("Инвентарный номер: ").strip()
if not inv_num:
    print("❌ Инвентарный номер обязателен!")
    input("Нажмите Enter...")
    return

name = input("Наименование: ").strip()
if not name:
    print("❌ Наименование обязательно!")
    input("Нажмите Enter...")
    return

serial = input("Серийный номер (Enter - пропустить): ").strip()
location = input("Местоположение (Enter - пропустить): ").strip()

# Выбор статуса
print("\nСтатусы:")
print("  1. На складе")
print("  2. В эксплуатации")
print("  3. В ремонте")
status_choice = input("Выберите статус (1-3, по умолчанию 1): ").strip()
status_map = {'1': 'in_stock', '2': 'in_use', '3': 'repair'}
status = status_map.get(status_choice, 'in_stock')

try:
    db.add_equipment(
        inventory_number=inv_num,
```

```
            name=name,
            serial_number=serial,
            status=status,
            location=location
        )
        print(f"\n✅ Оборудование '{name}' успешно добавлено!")
    except Exception as e:
        print(f"\n❌ Ошибка: {e}")

    input("\nНажмите Enter для продолжения...")

def view_equipment_menu():
    """Просмотр детальной информации об оборудовании"""
    clear_screen()
    print_header("ПРОСМОТР ОБОРУДОВАНИЯ")

    equipment = db.get_all_equipment()
    if not equipment:
        print("📭 Нет данных.")
        input("Нажмите Enter...")
        return

    print("Список оборудования:")
    for eq in equipment:
        print(f"  {eq[0]}. {eq[1]} - {eq[3]}")

    try:
        eq_id = int(input("\nВведите ID оборудования для просмотра: ").strip())
```

```
    eq = db.get_equipment_by_id(eq_id)
    if eq:
        print(f"\n--- Детальная информация ---")
        print(f"Инвентарный номер: {eq[1]}")
        print(f"Наименование: {eq[3]}")
        print(f"Серийный номер: {eq[2] or '-'}")
        print(f"Статус: {get_status_text(eq[5])}")
        print(f"Местоположение: {eq[6] or '-'}")
        print(f"Дата покупки: {eq[8] or '-'}")
        print(f"Гарантия до: {eq[9] or '-'}")
        print(f"Стоимость: {eq[10] if eq[10] else '-'}")

        # История перемещений
        history = db.get_equipment_history(eq_id)
        if history:
            print(f"\n--- История перемещений ---")
            for h in history:
                print(f"  {h[6]} | от: {h[7]} → к: {h[8] if h[8] else '-'} | {h[9]}")
    else:
        print("✗ Оборудование не найдено")
except ValueError:
    print("✗ Неверный ID")

input("\nНажмите Enter для продолжения...")
```

3. databade.py

(часть кода)

```
# database.py

import sqlite3

from datetime import datetime, timedelta

from io import BytesIO

import openpyxl

from openpyxl.styles import Font, Alignment, PatternFill


DB_NAME = 'equipment.db'


def get_connection():
    """Получить подключение к базе данных"""
    return sqlite3.connect(DB_NAME)



def init_db():
    """Создание всех таблиц и начальных данных"""
    with get_connection() as conn:
        cursor = conn.cursor()

        # Таблица пользователей
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT UNIQUE NOT NULL,
                password TEXT NOT NULL,
                role TEXT CHECK(role IN ('admin', 'it_user', 'mol')),
                full_name TEXT,
```

```
        department TEXT,
        email TEXT,
        created_at TEXT DEFAULT CURRENT_TIMESTAMP
    )
''')

# Таблица типов оборудования
cursor.execute('''
    CREATE TABLE IF NOT EXISTS equipment_types (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        category TEXT,
        depreciation_rate REAL
    )
''')

# Таблица оборудования
cursor.execute('''
    CREATE TABLE IF NOT EXISTS equipment (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        inventory_number TEXT UNIQUE NOT NULL,
        serial_number TEXT,
        name TEXT NOT NULL,
        type_id INTEGER,
        status TEXT DEFAULT 'in_stock',
        location TEXT,
        responsible_id INTEGER,
        purchase_date TEXT,
```

```
        warranty_end TEXT,
        cost REAL,
        qr_code TEXT,
        next_maintenance_date TEXT,
        created_at TEXT DEFAULT CURRENT_TIMESTAMP,
        updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (type_id) REFERENCES equipment_types(id),
        FOREIGN KEY (responsible_id) REFERENCES users(id)
    )
''')

# Таблица перемещений
cursor.execute('''
    CREATE TABLE IF NOT EXISTS movements (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        equipment_id INTEGER NOT NULL,
        from_mol_id INTEGER,
        to_mol_id INTEGER,
        from_location TEXT,
        to_location TEXT,
        date TEXT NOT NULL,
        document_number TEXT,
        created_at TEXT DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (equipment_id) REFERENCES equipment(id),
        FOREIGN KEY (from_mol_id) REFERENCES users(id),
        FOREIGN KEY (to_mol_id) REFERENCES users(id)
    )
''')
```

```
    # Таблица ремонтов
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS repairs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            equipment_id INTEGER NOT NULL,
            repair_date TEXT NOT NULL,
            description TEXT NOT NULL,
            cost REAL,
            FOREIGN KEY (equipment_id) REFERENCES equipment(id) ON DELETE CASCADE
        )
    ''')

    # Таблица лицензий
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS licenses (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            equipment_id INTEGER NOT NULL,
            software_name TEXT NOT NULL,
            license_key TEXT,
            expiration_date TEXT NOT NULL,
            FOREIGN KEY (equipment_id) REFERENCES equipment(id) ON DELETE CASCADE
        )
    ''')

    # Таблица расходных материалов
    cursor.execute('''
```

```
        CREATE TABLE IF NOT EXISTS consumables (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            compatible_models TEXT NOT NULL,
            quantity INTEGER DEFAULT 0,
            min_quantity INTEGER DEFAULT 5,
            unit TEXT DEFAULT 'шт',
            price REAL,
            last_updated TEXT DEFAULT CURRENT_TIMESTAMP
        )
    ''')

    # Таблица совместимости
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS compatibility (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            equipment_type_id INTEGER NOT NULL,
            component_type TEXT NOT NULL,
            compatible_part TEXT NOT NULL,
            description TEXT,
            FOREIGN KEY (equipment_type_id) REFERENCES equip-
ment_types(id)
        )
    ''')

    # Таблица истории списаний
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS consumables_usage (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            consumable_id INTEGER NOT NULL,
            equipment_id INTEGER,
            quantity INTEGER NOT NULL,
            usage_date TEXT DEFAULT CURRENT_TIMESTAMP,
            reason TEXT,
            FOREIGN KEY (consumable_id) REFERENCES consumables(id),
            FOREIGN KEY (equipment_id) REFERENCES equipment(id)
        )
    ''')

    # Начальные данные - пользователи
    cursor.execute("SELECT COUNT(*) FROM users")
    if cursor.fetchone()[0] == 0:
        users = [
            ('admin', 'admin123', 'admin', 'Администратор', 'IT-отдел', 'admin@company.ru'),
            ('ivanov', 'pass123', 'mol', 'Иванов Иван Иванович', 'Бухгалтерия', 'ivanov@company.ru'),
            ('petrova', 'pass123', 'mol', 'Петрова Анна Сергеевна', 'Отдел продаж', 'petrova@company.ru'),
            ('sidorov', 'pass123', 'it_user', 'Сидоров Дмитрий Николаевич', 'IT-отдел', 'sidorov@company.ru'),
            ('kozlov', 'pass123', 'mol', 'Козлов Михаил Андреевич', 'Отдел разработки', 'kozlov@company.ru'),
        ]
        cursor.executemany('''
            INSERT INTO users (username, password, role, full_name, department, email)
            VALUES (?, ?, ?, ?, ?, ?)
        ''', users)
```

```
print("  ✓ Добавлены пользователи")
```

ПРИЛОЖЕНИЕ Б. Скриншоты интерфейса

1. Система учета технических средств 1,2 страницы на сайте

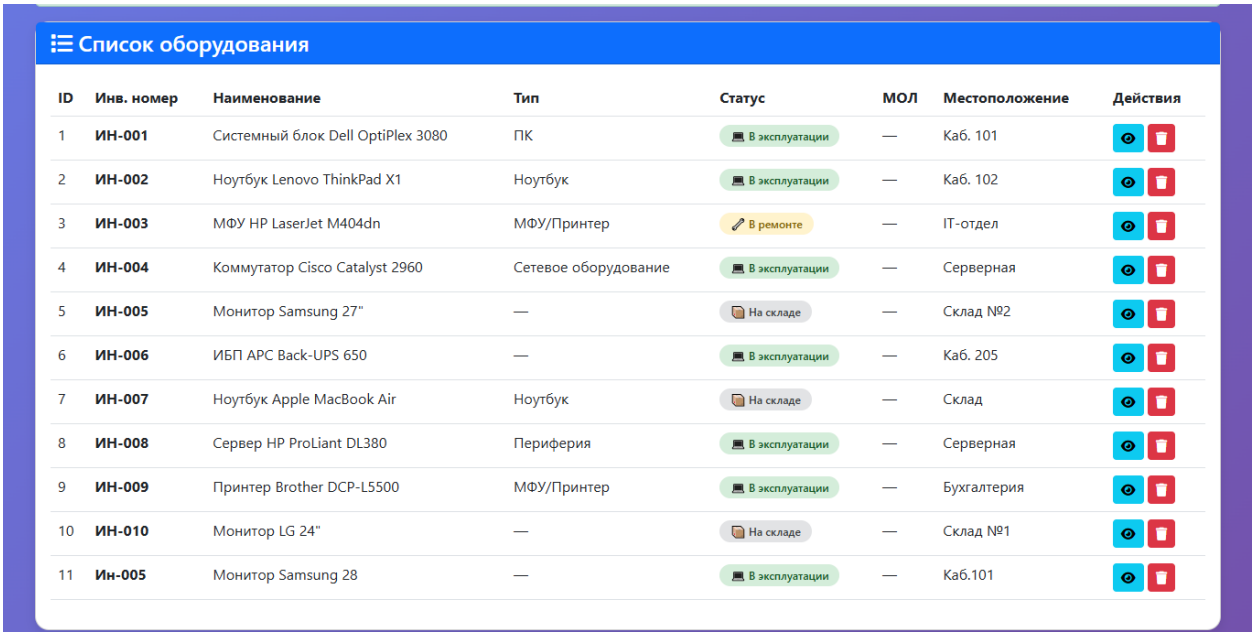


Рис 11. – список оборудования

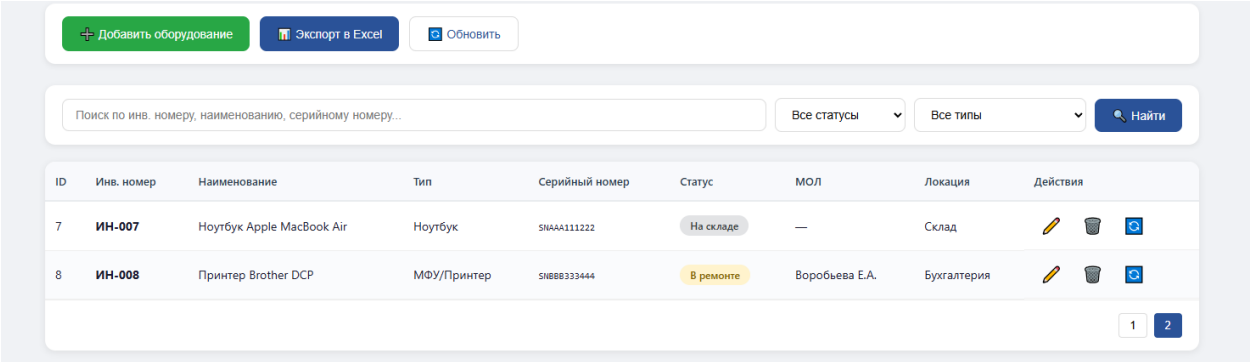


Рис 12. – просмотр оборудования

2. Добавление оборудования:

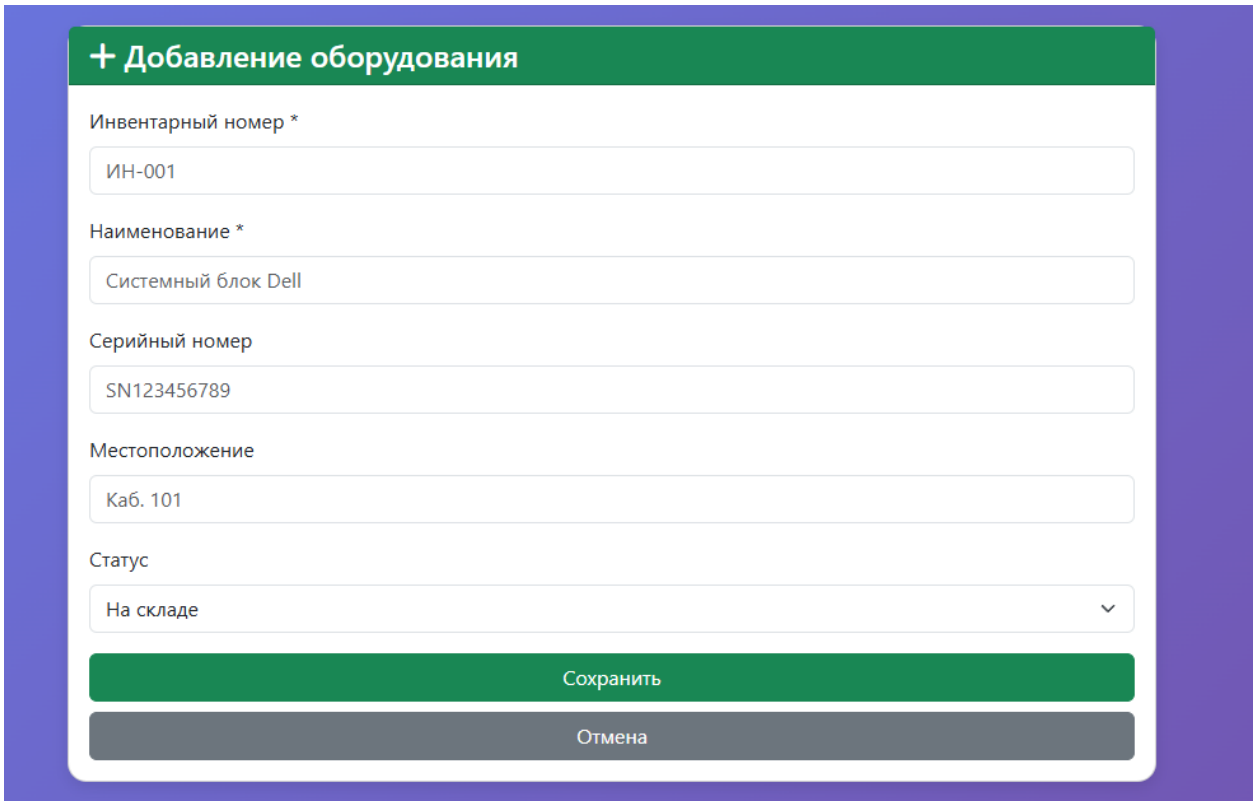


Рис 13. – добавление оборудования

3. Редактирование добавленного оборудования

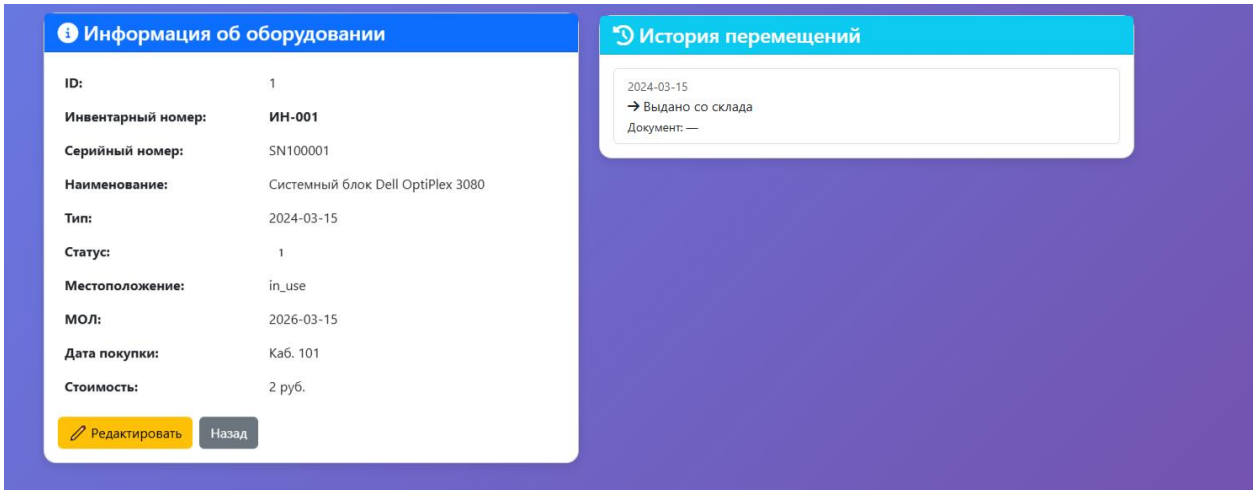


Рис 14. – редактирование оборудования

4. Вид в консоли

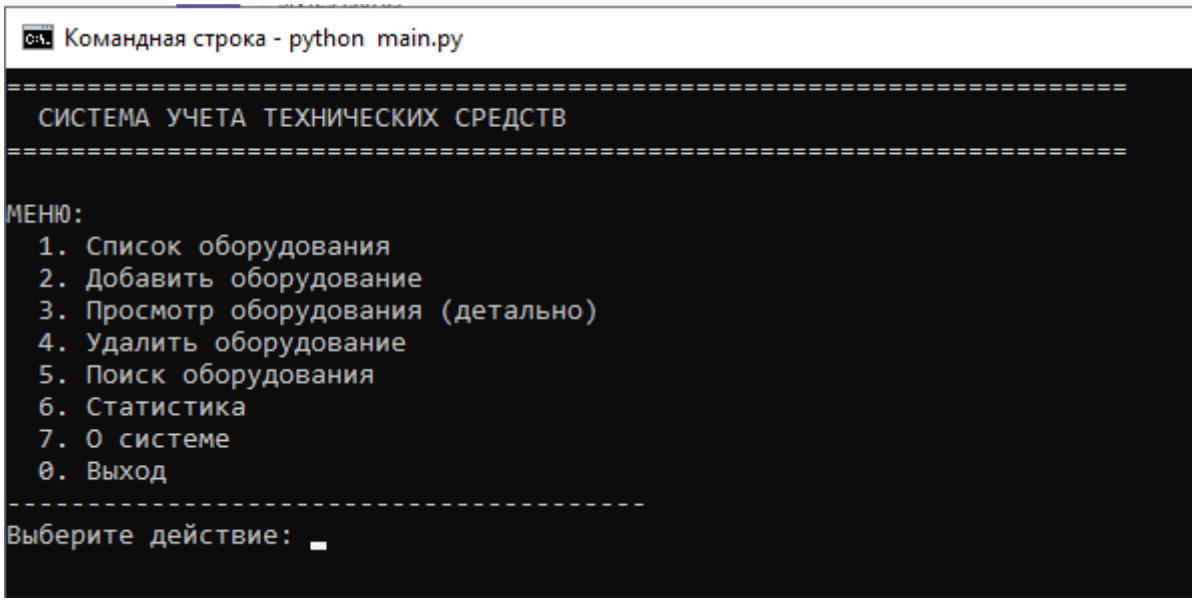


Рис 15. Система учета технических средств

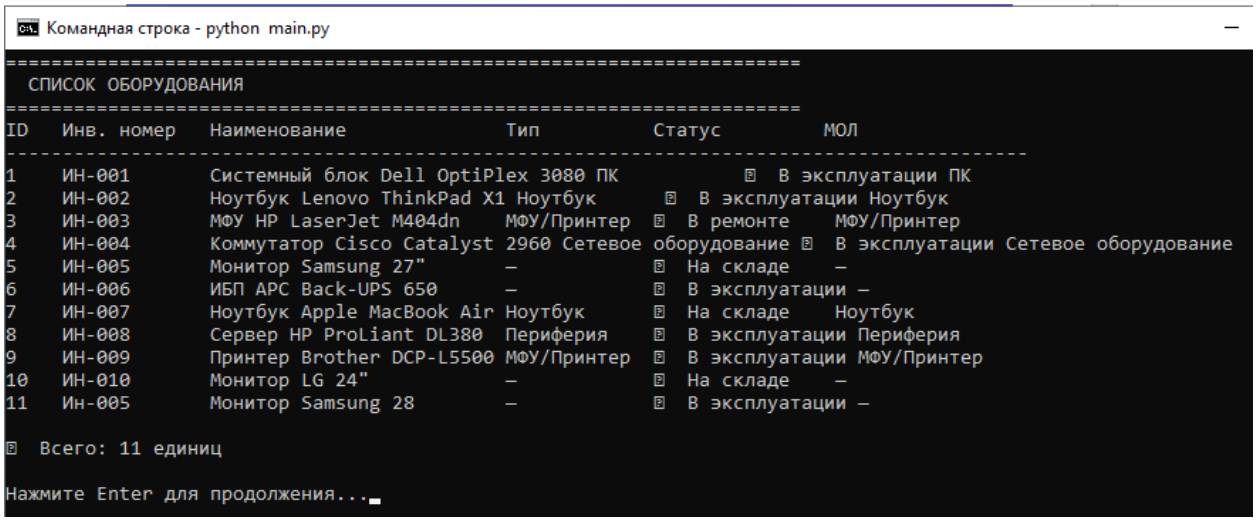


Рис 16. Список оборудования

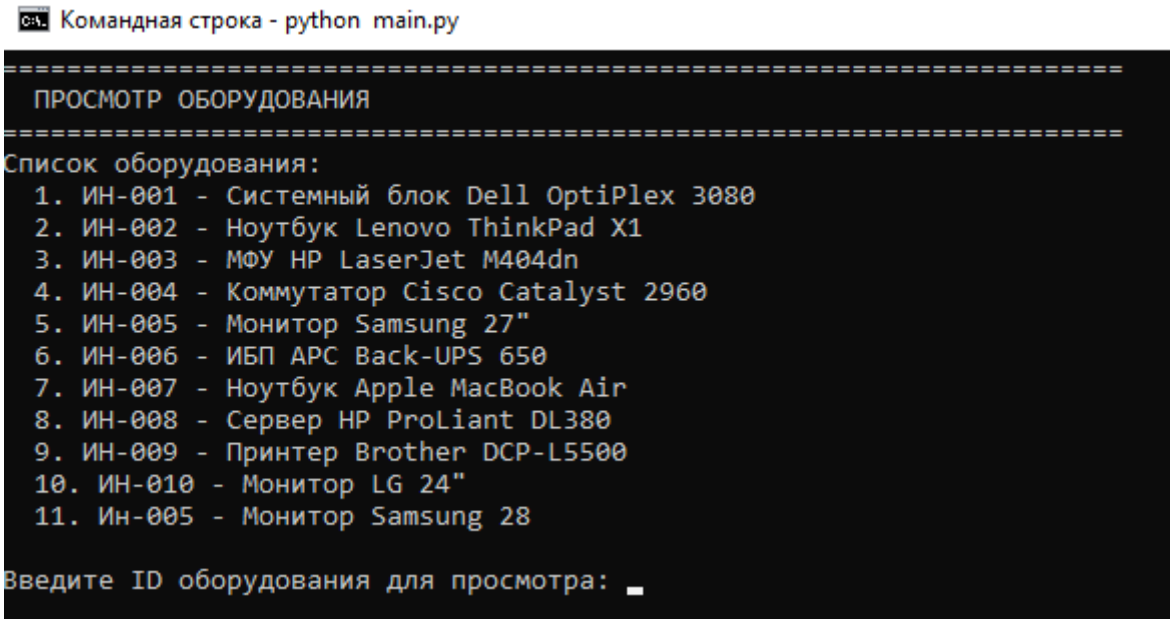


Рис 17. Просмотр оборудования

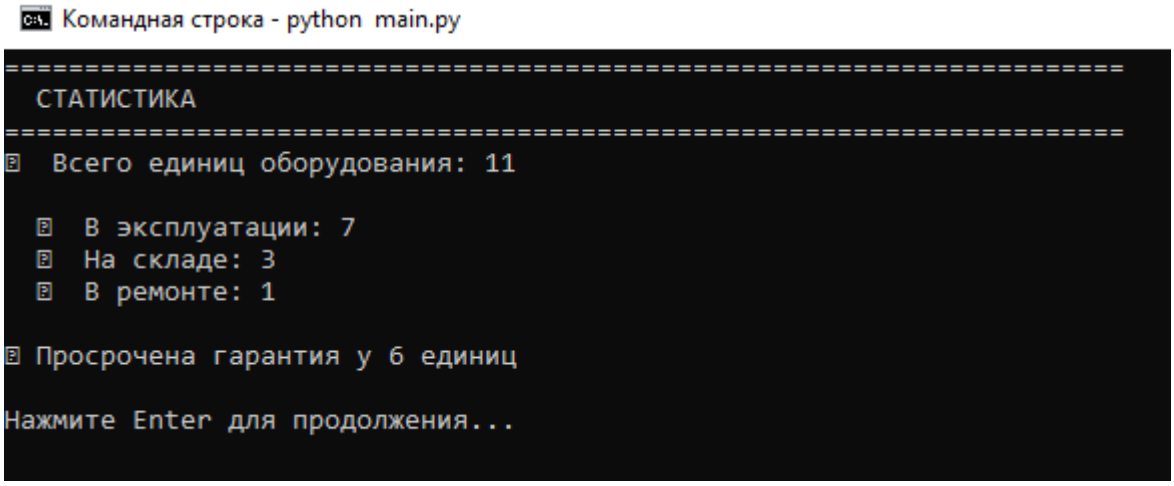


Рис 18. - статистика

Ссылка на сайт (не работает если не запущен сервер, app.py)
Остальные полные коды на переносном носителе.

http://127.0.0.1:5000